ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA EN SISTEMAS

DESARROLLO DE UN SISTEMA DE BÚSQUEDA Y ANÁLISIS DE SEGURIDAD DE DISPOSITIVOS IOT

DESARROLLO DEL COMPONENTE BACKEND

TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO COMO REQUISITO PARA LA OBTENCIÓN DEL TÍTULO DE INGENIERO DE SOFTWARE

MIKE ANGELO ENCALADA ONOFA
mike.encalada@epn.edu.ec

DIRECTOR: Dr. RICHARD PAUL RIVERA GUEVARA
richard.rivera01@epn.edu.ec

Quito, septiembre 2024

CERTIFICACIONES

Yo, Mike Angelo Encalada Onofa declaro que el trabajo de integración curricular aquí descrito es de mi autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que he consultado las referencias bibliográficas que se incluyen en este documento.

Mike Angelo Encalada Onofa

Certifico que el presente trabajo de integración curricular fue desarrollado por Mike Angelo Encalada Onofa, bajo mi supervisión.

Dr. Richard Paul Rivera Guevara

Director

DECLARACIÓN DE AUDITORÍA

A través de la presente declaración, afirmamos que el trabajo de integración curricular aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

AUTOR: MIKE ANGELO ENCALADA ONOFA

DIRECTOR: Dr. RICHARD PAUL RIVERA GUEVARA

COLABORADOR: ISAI JASSON SARANSIG ALMAGOR

DEDICATORIA

Quiero dedicar esta tesis a mis padres, quienes me han dado su apoyo incondicional a lo largo de mi vida. Gracias por creer en mí y por inculcarme el valor del esfuerzo y la perseverancia.

A mis amigos, Los Maincrafteros, por su comprensión y por ayudarme a mantener un equilibrio entre el estudio y la diversión.

También a mis profesores y mentores, quienes con su sabiduría y paciencia me han guiado a lo largo de este camino académico.

Sin todos ustedes, este logro no habría sido posible.

AGRADECIMIENTOS

Quiero expresar mi más sincero agradecimiento a la Escuela Politécnica Nacional por brindarme la oportunidad de formarme académicamente en un entorno de excelencia.

A mis padres, por su cariño y apoyo constante. Gracias por apoyarme en los momentos difíciles y por enseñarme a nunca rendirme en los momentos más difíciles.

A mis amigos, quienes han sido un pilar fundamental en esta etapa de mi vida. Gracias por su comprensión, paciencia y por ayudarme a encontrar el equilibrio entre el estudio y la vida social.

A mis profesores y mentores, por su sabiduría, paciencia y orientación a lo largo de mi carrera. Su guía ha sido invaluable para mi desarrollo académico y personal.

CONTENIDO

Resumen **1**

Abstract **2**

1 INTRODUCCIÓN **3**

1.1 Objetivos 4

1.1.1 Objetivo general 4

1.1.2 Objetivos específicos 5

1.2 Alcance 5

1.3 Marco teórico 6

1.3.1 Seguridad en el Internet de las cosas (IoT) 6

1.3.2 Modelo C4 8

1.4 Herramientas de desarrollo 9

1.4.1 Django 9

1.4.2 Django Rest Framework 10

1.4.3 Simple JWT 10

1.4.4 Bases de datos 10

1.4.5 Plataformas de desarrollo 11

1.4.6 Librerías útiles 11

2 METODOLOGÍA **12**

2.1 Marco de trabajo SCRUM 12

2.1.1 Equipo SCRUM 12

2.1.2 Eventos 13

2.1.3 Artefactos 13

2.2 Diagramas C4 14

2.2.1 Diagrama de contexto 14

2.2.2 Diagrama de contenedores 15

2.2.3 Diagrama de componentes 16

2.2.4 Diagrama de código 17

2.3 Mockups 18

2.4 Historias de Usuario 18

2.5 Product Backlog 19
2.6 Sprint Backlog 19
2.7 Sprints 19
2.7.1 Sprint 0 20
2.7.2 Sprint 1 23
2.7.3 Sprint 2 29
2.7.4 Sprint 3 32
2.7.5 Sprint 4 34
2.7.6 Sprint 5 38
2.8 Entregable Final 40

3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES 41
3.1 Resultados 41
3.1.1 Pruebas de funcionalidad 43
3.1.2 Pruebas de rendimiento 47
3.2 Conclusiones 48
3.3 Recomendaciones 49

4 REFERENCIAS BIBLIOGRÁFICAS 50

5 ANEXOS I

ÍNDICE DE FIGURAS

2.1 Diagrama de contexto 14
2.2 Diagrama de contenedores 15
2.3 Diagrama de componentes 16
2.4 Diagrama de código 17
2.5 Preparación del repositorio 21
2.6 Tembo 21
2.7 MongoDB Atlas 22
2.8 Lista de rangos de direcciones IP del sitio web IP2Location Lite 24
2.9 Ejemplo de un documento de la colección con los rangos de las direcciones IP de Ecuador. 26
2.10 Ejemplo de un documento de la colección con los dispositivos IoT escaneados. 26
2.11 Ejemplo de documentos de la colección con los puertos y su información. . . 27
2.12 Resultado de la primera API de HU005. Obtener los rangos de direcciones IP del territorio ecuatoriano. 27
2.13 Resultado de la segunda API de HU010. Obtener la información relevante de un puerto, puerto 443. 28
2.14 Frame del dashoboard en Figma. 30
2.15 Consulta para obtener la información necesaria del mapa de calor. 31
2.16 Resultado de la API de HU008. Servicio para obtener la información de los diagramas del dashboard. 32
2.17 Resultado de la API del HU006. Servicio para re escanear una dirección IP. . 34
2.18 Ejemplo del correo electrónico enviado para la confirmación y activación de la cuenta de usuario. 36
2.19 Resultado de una de las API del HU001. Servicio para iniciar sesión y obtener los tokens de acceso. 37
2.20 Resultado de la API del HU009. Servicio para generar y obtener la imagen de la captura de pantalla de un servicio web. 37
2.21 Resultado de una de las API del HU003. Servicio para obtener todos los usuarios registrados. 40

3.1 Ejemplo de prueba automatizada en Postman para el servicio API RESTful de la captura de pantalla de un servicio web. 44
3.2 Resultado de la prueba de rendimiento hecha al servicio API RESTful con id API23. 47

ÍNDICE DE TABLAS

1.1 Plataformas de desarrollo 11
1.2 Librerías útiles 11

2.1 Equipo SCRUM 12
2.2 Eventos SCRUM 13
2.3 Artefactos SCRUM 13
2.4 Requisitos para el Sprint 1 23
2.5 Requisitos para el Sprint 2 29
2.6 Requisitos para el Sprint 3 33
2.7 Requisitos para el Sprint 4 35
2.8 Requisitos para el Sprint 5 38

3.1 Catálogo de todos los servicios API RESTful creados en el compontente back end 41
3.2 Resultados de la prueba de funcionalidad del servicio API RESTful con id API23 de la historia de usuario HU010 44
3.3 Resultados de la prueba de funcionalidad del servicio API RESTful con id API7 de la historia de usuario HU008 45
3.4 Resultados de la prueba de funcionalidad del servicio API RESTful con id API18 de la historia de usuario HU009 45
3.5 Resultados de la prueba de funcionalidad del servicio API RESTful con id API1 de la historia de usuario HU001 46
3.6 Resultados de la prueba de funcionalidad del servicio API RESTful con id API2 de la historia de usuario HU001 46
3.7 Resultados de la prueba de rendimiento del servicio API RESTful con id API23 de la historia de usuario HU010 48

5.1 Product Backlog I
5.2 Historia de usuario HU001 II
5.3 Historia de usuario HU002 III
5.4 Historia de usuario HU003 III
5.5 Historia de usuario HU004 IV

5.6 Historia de usuario HU005 V
5.7 Historia de usuario HU006 V
5.8 Historia de usuario HU007 VI
5.9 Historia de usuario HU008 VII
5.10 Historia de usuario HU009 VII
5.11 Historia de usuario HU010 VIII
5.12 Caso de prueba CP_API_001 IX
5.13 Caso de prueba CP_API_002 XI
5.14 Caso de prueba CP_API_003 XIII
5.15 Caso de prueba CP_API_004 XV
5.16 Caso de prueba CP_API_005 XVII

RESUMEN

Los dispositivos IoT han tenido una gran popularidad en los últimos años, por lo que su adopción ha ido aumentando día tras día, existe una amplia variedad de dispositivos IoT utilizados en diferentes áreas, en salud, en el hogar, en el transporte, etc., lo que significa que estos dispositivos tienen alcances distintos, con funcionalidades y recursos limitados. Estas limitaciones son una de las razones principales por lo que estos dispositivos son inseguros, al contar con recursos limitados las medidas de seguridad quedan en segundo plano enfocándose los proveedores en las funciones principales del dispositivo, a su vez, estos proveedores no suelen tener una capacitación adecuada con respecto a la seguridad en los sistemas IoT.

La falta de medidas de seguridad junto con la amplia adopción de estos dispositivos IoT en los último días, son pruebas suficientes de la necesidad de un sistema para mitigar este problema. El objetivo de este proyecto es el desarrollo de un sistema que ayude a detectar posibles vulnerabilidades en dispositivos IoT, con el fin de que varios propietarios de estos dispositivos puedan buscar y encontrar posibles vulnerabilidades, con el fin de que los propietarios puedan solventar estos riesgos de seguridad.

Keywords: IoT, Vulnerabilidades, Sistema, Seguridad

ABSTRACT

IoT devices have been very popular in recent years, so their adoption has been increasing day by day, there is a wide variety of IoT devices used in different areas, in health, home, transportation, etc., which means that these devices have different scopes, with limited functionalities and resources. These limitations are one of the main reasons why these devices are insecure, as they have limited resources, security measures are put on the back burner and vendors focus on the main functions of the device, at the same time, these vendors usually do not have adequate training regarding security in IoT systems.

The lack of security measures along with the widespread adoption of these IoT devices in recent days, are sufficient evidence of the need for a system to mitigate this problem. The objective of this project is to develop a system to help detect potential vulnerabilities in IoT devices, so that various owners of these devices can search for and find potential vulnerabilities, in order for the owners to address these security risks.

Keywords: IoT, Vulnerabilities, System, Security

1 INTRODUCCIÓN

Los dispositivos IoT o Internet de las cosas en los últimos tiempos han mejorado la calidad de vida de muchas personas porque proporcionan varios beneficios en la vida cotidiana [1], convirtiéndose, en muchos casos, en un elemento fundamental en la vida diaria, un ejemplo de estos beneficios se demuestra en la mejora de seguridad y protección, usuarios comunes son capaces de crear o actualizar un sistema de seguridad para sus hogares a través de dispositivos IoT (cámaras IP, cerraduras inteligentes o sensores de movimiento) logrando un monitoreo, y por ende, una captura de datos a tiempo real como un método de vigilancia inmediata. Otro beneficio se evidencia en la mejora en la automatización de las tareas rutinarias dentro del hogar, con dispositivos IoT (bombillas, termostatos y enchufes inteligentes) se logra de manera automática realizar actividades diarias como encender las luces, abrir las cortinas o ajustar el termostato.

La implementación de dispositivos IoT está aumentando rápidamente debido a los beneficios antes mencionados con muchos otros beneficios adicionales, y se encuentra en varias áreas, esto se lo puede evidenciar a través de la clasificación de estos dispositivos: IoT Industrial, IoT Comercial, IoT para el cuidado de la salud, IoT de transporte y IoT de consumo [2], la pandemia del COVID-19, a su vez, ha impulsado el uso de estos dispositivos, presionando a las personas a realizar arreglos para el trabajo remoto, donde los dispositivos IoT resultaron útiles para realizar configuraciones en los entornos de trabajo [3] dentro del hogar, el monitorea miento de la salud de las personas se volvió también algo crucial en ese periodo, como consecuencia se recurrió a los dispositivos IoT para realizar este seguimiento a tiempo real. Así entonces, la aplicación de IoT genera varios beneficios, aumentando drásticamente su implementación, pero a su vez crea y aumenta los riesgos debido a la falta de medidas de seguridad que tienen estos dispositivos.

En Ecuador, la adopción de tecnologías emergentes es bastante alta, y esto se está acelerando debido a empresas que buscan hacer uso de estas tecnologías para generar un

impacto positivo en los negocios, según una encuesta realizada por Building a Better Working World en el año 2022 podemos darnos cuenta de que la tecnología de IoT ocupa el puesto número 6 como tecnología emergente con mayor posibilidad de adopción a corto plazo [4]. El incremento en la implementación de dispositivos IoT, junto con la gran cantidad de potentes riesgos de seguridad que esto conlleva, como el uso de estos dispositivos para distribuir malware [5] o su uso para espionaje [6] es evidencia suficiente de la urgente necesidad de tomar medidas con el fin de mitigar los desafíos de seguridad asociados con estos dispositivos.

El propósito general de este proyecto es el desarrollo de un sistema web, implementando las mejores prácticas dentro de su ciclo de desarrollo, con el fin de evaluar los diferentes dispositivos IoT presentes en el Ecuador, identificando y analizando las posibles vulnerabilidades que estos presentan, por lo que se busca proveer una evaluación con respecto a las vulnerabilidades presentes en los dispositivos, contribuyendo a mejorar la seguridad de estos dispositivos IoT dentro del territorio Ecuatoriano.

La estructura del presente documento inicia con la sección 1.0 comenzando con la introducción seguido de los objetivos, ahí se plantea el objetivo general y los objetivos específicos, también se presenta el alcance y el marco teórico referente al proyecto. En la sección 2.0 metodología se describe el marco de trabajo Scrum implementado en el desarrollo del sistema. Por último, en la sección 3.0 se presenta los resultados, conclusiones y recomendaciones que se obtuvieron al finalizar el proyecto.

1.1 OBJETIVOS

En esta sección se presentarán los objetivos del componente backend del proyecto, aquí se expondrá tanto el objetivo general como los objetivos específicos.

1.1.1 Objetivo general

El objetivo general del componente consiste en desarrollar un sistema software para la búsqueda y análisis de vulnerabilidades presentes en dispositivos IoT dentro del territorio ecuatoriano, enfocándose principalmente en el diseño e implementación del sistema y siguiendo buenas prácticas de desarrollo de software. El componente se lo terminará dentro

del plazo de tiempo definido, con los resultados finales esperados.

1.1.2 Objetivos específicos

Desarrollar el componente encargado de realizar la recolección, gestión y procesamiento de la información recopilada durante la búsqueda de los dispositivos IoT y el análisis de sus potenciales vulnerabilidades.

Implementar medidas de seguridad con el fin de proteger información sensible y prevenir posibles amenazas que puedan afectar a la privacidad del usuario, garantizando la integridad, consistencia y seguridad de los datos.

Establecer una comunicación efectiva con el componente Frontend para presentar de manera más organizada y legible los datos procesados dentro de un dashboard web.

1.2 ALCANCE

El alcance del desarrollo del componente Backend dentro del proyecto incluirá desde la fase de recopilación y análisis de requisitos hasta la implementación final del sistema, teniendo en consideración la escalabilidad y robustez del sistema a lo largo de su ciclo de desarrollo, además se hará uso de herramientas de desarrollo de software actuales y modernas, incluyendo entornos de desarrollo integrados (IDE), herramientas para el control de versiones, herramientas para pruebas automatizadas, entre otros.

En la fase de diseño se incluirá el modelo de arquitectura de software C4, este modelo permitirá representar la arquitectura total del sistema de una manera más simple de entender, lo que permitirá comunicar eficientemente el estado del sistema a cualquier interesado.

Al final del proyecto, se espera implementar exitosamente el sistema y obtener la documentación final que describirá todo el ciclo de desarrollo de este, teniendo como entregables finales del proyecto, el sistema de búsqueda y análisis de seguridad de dispositivos IoT totalmente funcional y el documento final del trabajo de integración curricular.

1.3 MARCO TEÓRICO

En la siguiente sección se abarca un conjunto de conceptos y definiciones importantes que se tendrán en cuenta a lo largo del desarrollo del proyecto.

1.3.1 Seguridad en el Internet de las cosas (IoT)

El Internet de las cosas o IoT se refiere a una red dispositivos físicos, llamados dispositivos IoT, estos dispositivos son capaces de transferirse información entre sí sin la necesidad de la interacción con una persona [7], a través de sus mecanismos embebidos, sensores, software y conectividad de red, estos dispositivos, a diferencia de las computadoras tradicionales, tienen funciones específicas y recursos limitados, por lo que existen una gran variedad de estos, desde dispositivos para hogares inteligentes, como enchufes, bombillas o cortinas inteligentes, pasando por dispositivos portables como smartwatch, hasta dispositivos industriales y para el transporte [8].

Los dispositivos IoT tienen una gran popularidad por lo que existe un gran incremento de estos dispositivos IoT haciendo necesario el enfoque en la seguridad debido a las pocas medidas de protección con las que cuentan [9].

Los dispositivos IoT son particularmente propensos a la piratería y malware dirigido según la Unidad de Ciberseguridad del Departamento de Justicia de los Estados Unidos. Los dispositivos IoT por lo general tiene muchos defectos y cuentan con medidas de seguridad mínima [2], y a pesar de eso la implementación de estos sigue aumentando cada día, permitiendo que todos estos dispositivos inseguros se encuentren conectados al Internet, varios de estos dispositivos cuentan con funciones integradas, cámaras, micrófonos y sensores, por lo que recopilan diariamente demasiada información, en algunos casos sin que el usuario este consiente de esto, así entonces, esta información puede terminar en manos de alguna persona mal intencionada, recopilando, distribuyendo y divulgando esta información, afectando a la privacidad del usuario [10].

Los recursos informáticos de estos dispositivos por lo general son limitados lo que provoca que cada dispositivo tenga funciones específicas, propósitos de uso y estructuras físicas diferentes, lo que resulta en una gran diversidad de dispositivos IoT, esta diversidad es la razón principal de la falta de seguridad en estos dispositivos, varios fabricantes y desarro-

lladores de tecnología IoT no tienen la formación necesaria con respecto a los aspectos de seguridad para cada uno de los dispositivos, la limitada capacidad y recursos de los dispositivos, también dificulta la implementación de medidas de seguridad actuales [2].

El Proyecto de Seguridad de Aplicaciones Web Abiertas (OWASP) que inició en el año 2001, con el fin de exponer y, a la vez, comprender los riesgos de seguridad del software, actualiza regularmente las 10 vulnerabilidades más importantes o frecuentes que pueden afectar a los sistemas software, adicionalmente detalla también las 10 vulnerabilidades más importantes que pueden afectar a los dispositivos IoT, la última actualización de esta lista se lo realizó en el año 2018, algunas vulnerabilidades de esta lista son:

1. **Contraseñas débiles, adivinables o codificadas. –** Los dispositivos IoT frecuentemente usan credenciales que pueden ser fácilmente violentadas, están disponibles públicamente o no pueden ser actualizadas [11].
2. **Servicios de red inseguros. –** Los dispositivos IoT pueden estar ejecutando servicios de red innecesarios o inseguros expuestos a Internet, permitiendo el acceso a personas no autorizadas [11].
3. **Falta de mecanismo de actualización seguro. –** Algunos dispositivos puede estar ejecutando versiones de firmware obsoleto o vulnerable, debido a la falta de notificaciones de cambios de seguridad, falta de validación o envío inseguro del firmware [11].

La seguridad en los dispositivos IoT se encarga, a través de una estrategia de ciberseguridad, de proteger estos dispositivos y a las redes vulnerables a las cuales se conectan [12]. La seguridad de dispositivos IoT es importante, debido a que muchos de estos dispositivos no cuentan con mecanismos de seguridad, realizan el manejo de información sensible y envían de manera insegura información a través de la red [13].

El escaneo de puertos es una herramienta útil para detectar vulnerabilidades, los puertos permiten a las computadores utilizar una conexión de red para gestionar solicitudes entrantes y salientes [14]. Estos puertos están siempre definidos y se asocian con un tipo específico de servicio [15], al detectar si los puertos están abiertos, activos o habilitados, entonces se puede conocer el posible servicio o protocolo, logrando analizar si el servicio es conforme al dispositivo IoT asociado y conocer las posibles vulnerabilidades que este puede presentar.

Así entonces, existe un gran conjunto de aspectos de seguridad que se deben tener en cuenta para los dispositivos IoT como la supervisión administrativa, elaboración de parches, actualizaciones regulares, uso de contraseñas seguras y seguridad en la red [12].

1.3.2 Modelo C4

El modelo de arquitectura de software C4, desarrollado por Simon Brown, proporciona una vista general de un sistema software de una forma sencilla, escalable y flexible [16], por lo que ayuda a los arquitectos y desarrolladores de software a comunicar la arquitectura del sistema de una manera más eficiente y entendible para cualquier persona, en otras industrias como la de diseño y construcción de edificios los planos presentados a los clientes son entendibles, es decir, el cliente es capaz de percibir correctamente la vista general de la estructura diseñada, en cambio, en el diseño y desarrollo de software los modelos arquitectónicos no son entendibles para los usuarios finales u otros miembros del equipo de desarrollo, por lo que esto impide su correcta comunicación, causando así, problemas en la construcción del sistema.

El modelo C4 se creó con el fin de resolver este problema, el modelo presenta 4 niveles de detalle yendo de lo más general a lo más particular, estos niveles son:

1. **Contexto. -** Este nivel es el más general, presenta una vista panorámica del sistema, mostrando los límites del sistema, los usuarios y su interacción con el sistema [17].
2. **Contenedor. -** Este nivel detalla los componentes individuales del sistema, servidores web, bases de datos, aplicaciones o servicios, y su conexión entre estos [17].
3. **Componente. -** En este nivel se descomponen aún más cada contenedor, se identifican sus bloques de construcción estructurales (componentes) y sus respectivas interacciones [18].
4. **Código. -** Este nivel es el más particular, en este se describe la estructura de código de cada componente, las clases, los módulos y los paquetes o librerías [17].

Así entonces, cada nivel del modelo C4 es creado para el entendimiento y comprensión de cada miembro del proyecto. Los usuarios finales y los altos mandos de la organización pueden hacer uso del nivel de contexto y del contenedor (detalles de alto nivel) para examinar la estructura del sistema, en cambio, los desarrolladores de software hacen uso del

nivel componentes y código (detalles de bajo nivel) [17] para guiarse durante la creación del sistema.

1.4 HERRAMIENTAS DE DESARROLLO

En esta sección se explicará detalladamente las herramientas de desarrollo principales utilizadas para la realización de este proyecto.

1.4.1 Django

Django es un framework del lado del servidor para el desarrollo de aplicaciones web de manera rápida y simple, gracias a la amplia colección de módulos reutilizables que ofrece este framework, permitiendo el uso de varias funcionalidades que se desearía tener en una aplicación web como la autenticación, gestión de cookies, acceso a la base de datos, envío de correos electrónicos, entre otros, sin la necesidad de codificarlos como en otros frameworks o herramientas.

Las aplicaciones construidas en Django se basan en la arquitectura Modelo-Vista-Template para gestionar el sistema de solicitud-respuesta [19].

1. **Modelo.-** Permiten construir o mapear las tablas de datos a través de clases y sus registros a través de objetos de esas clases, esto conocido como ORM o Mapeo Objeto Relacional, permitiendo el acceso y manipulación de los registros de cada tabla de manera simple.

2. **Vista.-** Se encargan de manejar las solicitudes realizadas por los usuarios, es decir, dependiendo de la solicitud este decide que respuesta entregar a cada usuario, haciendo uso de los modelos o no.

3. **Template.-** Administran la presentación de la información que se visualizará en el navegador web.

El uso del framework Django tiene varías ventajas importantes para el desarrollo de este proyecto, como la rapidez en el desarrollo, implementación de seguridad, extrema escalabilidad y su amplia colección de módulos.

1.4.2 Django Rest Framework

Django Rest Framework es una de las varias librerías o módulos externos que se pueden integrar a Django, este módulo ofrece herramientas poderosas y flexibles para la creación de servicios API RESTful.

El uso de este módulo en el proyecto permitirá la creación de servicios API RESTful de una manera simple y rápida, permitiendo el intercambio efectivo de información entre el back end y el front end.

1.4.3 Simple JWT

Simple JWT es un módulo o librería externa que se puede integrar con Django y a su vez con el otro módulo Django Rest Framework, este módulo provee la autenticación a través de Json Web Token en Django, específicamente, para el consumo de los servicios API Rest.

El uso de Simple JWT en el proyecto permitirá tener un intercambio seguro de información entre el back end y el front end, asegurando la disponibilidad, integridad y confidencialidad de los datos del usuario.

1.4.4 Bases de datos

Para el proyecto se decidió utilizar dos bases de datos, Postgresql y Mongodb, la base de datos relacional Postgresql contendrá la información de los usuarios del sistema, registros, grupos, privilegios, entre otros., y la base de datos no relacional MongoDB contendrá la información relacionado a los dispositvos IoT, rangos de IP e información de los puertos.

Se decidió optar por el uso de dos bases de datos debido a algunas peculiaridades encontradas al inicio del desarrollo, Django al no ser 100 % compatible con las bases de datos no relacionales complicaba su integración, pero el uso de MongoDB, una base de datos no relacional basada en documento, era necesaria por la eficiencia que tenía al momento de realizar consultas en una cantidad de datos muy grande.

Así entonces, la información relacionada al usuario era manejada por el ORM de Django, mapeando las tablas en la base de datos relacional Postgresql, y la información de los dispositivos IoT serán manejados con consultas propias de mongoDB con pymongo.

1.4.5 Plataformas de desarrollo

A lo largo del proyecto se hizo uso de plataformas de desarrollo como IDEs, gestores de base de datos o plataformas para el consumo de APIs, las cuales se detallan en la Tabla 1.1.

Tabla 1.1: Plataformas de desarrollo

Plataforma	Descripción
Visual Studio	Editor de texto usado como entorno de desarrollo o programación.
Mongo Compass	Gestor de la base de datos MongoDB usado para visualizar la información de los dispositivos IoT durante el desarrollo.
PgAdmin	Gestor de la base de datos Postgresql usado para visualizar la información de los usuarios durante el desarrollo.
Postman	Plataforma para consumir y visualizar la información relacionada a las APIs creadas durante el desarrollo.

1.4.6 Librerías útiles

Para el desarrollo del sistema se hizo uso de algunos módulos o librerías de python para la obtención y manejo de datos dentro del programa, estas librerías se detallan en la Tabla 1.2

Tabla 1.2: Librerías útiles

Librería	Descripción
geoip2	Útil para obtener la información como el país, provincia o ciudad de una dirección IPv4.
pyppeteer	Útil para obtener las capturas de pantalla de un servicio web, a través de una dirección IPv4 o de un nombre de dominio.
ipaddress	Útil para el manejo de direcciones IPv4, permitiendo realizar operaciones con estas, como sumas o restas.
pillow	Para el manejo de imágenes, ofrece funcionalidades como abrir y guardar imágenes con diferentes formatos.

2 METODOLOGÍA

2.1 MARCO DE TRABAJO SCRUM

SCRUM será el marco de trabajo utilizado para el desarrollo del sistema, esto debido a la familiaridad de los miembros del equipo con este marco de trabajo, al tener experiencia con su implementación resultará más fácil su uso, reduciendo los riesgos que existirían al adoptar una metodología desconocida.

El marco de trabajo SCRUM además ofrece algunos beneficios deseados para el desarrollo del sistema, como las entregas continuas o incrementos del producto lo que permite al usuario dar un feedback continuo sobre su opinión del producto dando la oportunidad a los desarrolladores de realizar mejoras o cambios, otro beneficio importante es la mejora en el control del trabajo, SCRUM, al dividir el proyecto en sprints junto con sus famosas reuniones o meetings, permite tener el control de los avances que se hacen en el producto.

2.1.1 Equipo SCRUM

El equipo SCRUM está presente durante todo el ciclo de desarrollo del sistema, son los que se encargan de dar valor al producto en cada uno de las fases o ciclos conocidos como sprints, Scrum describe a este equipo como auto gestionado y multifuncional, utilizando la retroalimentación para mejorar constantemente [20] , este equipo se detalla en la Tabla 2.1.

Tabla 2.1: Equipo SCRUM

Rol	**Responsable**
Product Owner	Ing. Richard Rivera
Developers	Mike Encalada - Isai Saransig (Componente Backend y Frontend respectivamente)
Scrum Master	Ing. Richard Rivera

2.1.2 Eventos

Los eventos SCRUM son sucesos predeterminados que ocurren a lo largo del desarrollo, estos eventos ayudan a tener un control en el desarrollo efectivo del producto, estos eventos se detallan en la Tabla 2.2.

Tabla 2.2: Eventos SCRUM

Evento	Descripción
Sprint	Ciclo de desarrollo o periodo de tiempo de trabajo que dura, en este caso, entre 1 o 2 semanas.
Sprint Planning	Reunión antes de realizar un sprint para discutir sobre los objetivos del próximo sprint, dura entre 30 o 45 minutos.
Daily Scrum	Reunión diaria para conocer el avance del producto, dura entre 15 o 30 minutos.
Sptrint Review	Reunión para verificar los resultados deseados en cada sprint, dura entre 1 o 2 horas.
Sptrint Retrospective	Reunión para hablar sobre posibles problemas o inconvenientes suscitados en cada sprints, con el fin de mejorar en los próximos sprints, dura entre 30 o 45 minutos.

2.1.3 Artefactos

Los artefactos Scrum son elementos físicos y formales que representan el trabajo o valor en cada Sprint [21], sirve como guía para el equipo Scrum, los artefactos se detallan en la Tabla 2.3.

Tabla 2.3: Artefactos SCRUM

Artefacto	Descripción
Product Backlog	Lista todos los requerimientos o tareas que se deben añadir en cada Sprint, detallándolos con su prioridad y su orden.
Sprint Backlog	Clasifica cada tarea del product backlog por Sprints, en este caso, se tiene un total de 5 sprints.
Incremento	Es el producto funcional obtenido en cada Sprint.

2.2 DIAGRAMAS C4

Los diagramas C4 son elementos adicionales al marco de trabajo SCRUM que se ocuparán para describir gráficamente el sistema, estos diagramas permiten representar al sistema en diferentes perspectivas, yendo desde lo más general hasta lo más particular, con el diagrama de contexto siendo el diagrama con la perspectiva más general hasta el diagrama de código siendo el más particular, siendo en total 4 diagramas, esto permite comunicar el sistema a diferentes tipos de personas, como el usuario que por lo general no tiene conocimientos técnicos de desarrollo de software, hasta al equipo de desarrollo que se basa en los diagramas al momento de la construcción del sistema.

2.2.1 Diagrama de contexto

El diagrama de contexto, como se mencionó anteriormente, es el diagrama de la perspectiva más general del sistema, en donde se detallan los usuarios y los sistemas que intervienen, el sistema principal o a desarrollar y los posibles sistemas externos, como se aprecia en la Figura 2.1, este diagrama es el más entendible por parte del usuario.

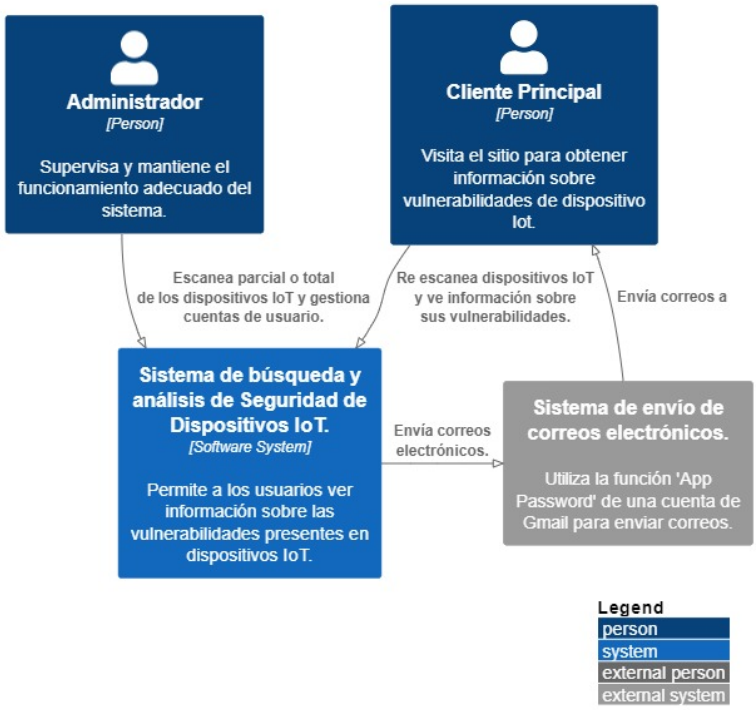


Figura 2.1: Diagrama de contexto

En el diagrama se detalla, primero a los dos tipos de usuarios que harán uso del sistema, el Administrador y el Cliente Principal, después tenemos el sistema a desarrollar, Sistema de búsqueda y análisis de Seguridad de Dispositivos IoT, y por último, un sistema externo, Sistema de envío de correos electrónicos, cada uno de estos elementos relacionados y detallados textualmente en el diagrama.

2.2.2 Diagrama de contenedores

El diagrama de contenedores describe el sistema principal o a desarrollar a un nivel más profundo y detallado, donde se puede apreciar las aplicaciones o contenedores que forman al sistema, parecido a un típico diagrama de arquitectura, como se puede ver en la Figura 2.2.

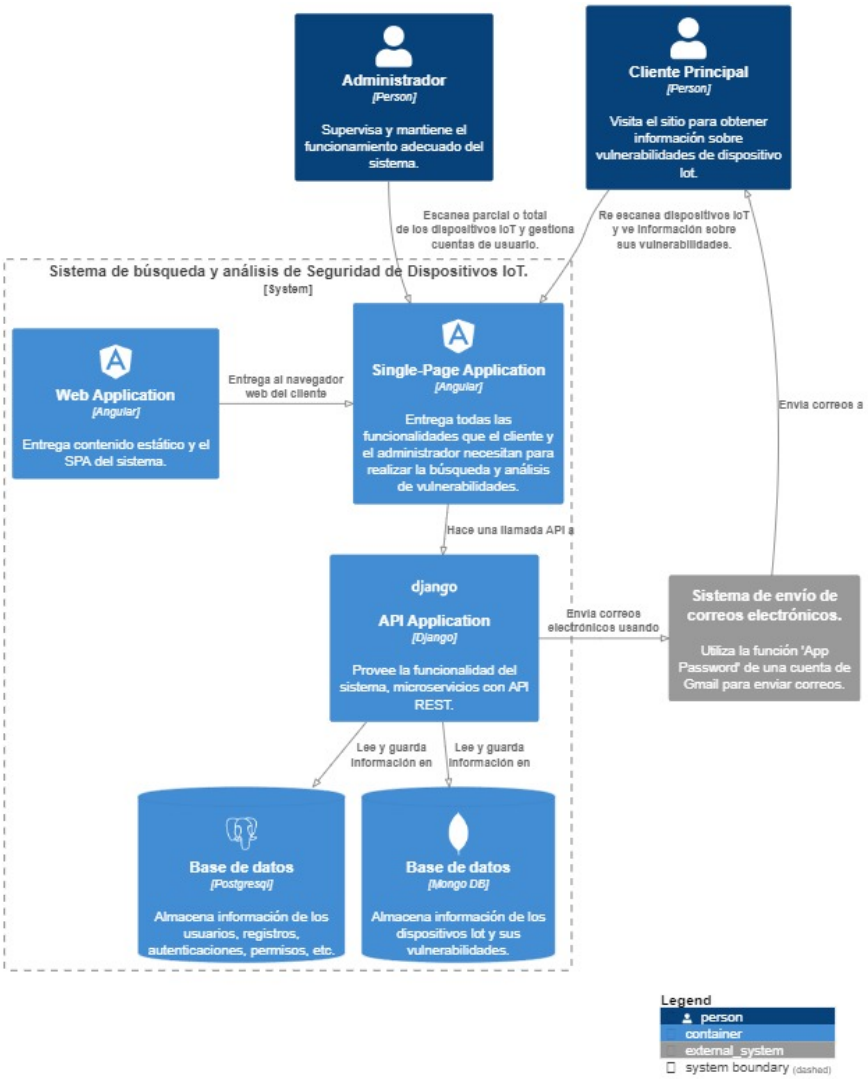


Figura 2.2: Diagrama de contenedores

En el diagrama se puede apreciar como se describe el sistema principal, podemos observar que para el desarrollo del sistema se hará uso de 4 aplicaciones o contenedores, Angular para el front end, Django para el back end y por último las dos bases de datos, Postgresql y MongoDB. Angular se describe como la aplicación que hará el consumo de los servicios API RESTful suministrados por Django, y Django a su vez accederá a las bases de datos para la realización de lectura y escritura de la información.

2.2.3 Diagrama de componentes

El diagrama de componentes describe a profundidad o con más detalle una de las aplicaciones o contenedores, en este caso se describe únicamente la aplicación desarrollada en Django, aquí se describe en componentes, que son las responsabilidades y los detalles de la tecnología o implementación [18], básicamente se separa el software en módulos, en este caso se divide en aplicaciones de Django como se ve en la Figura 2.3.

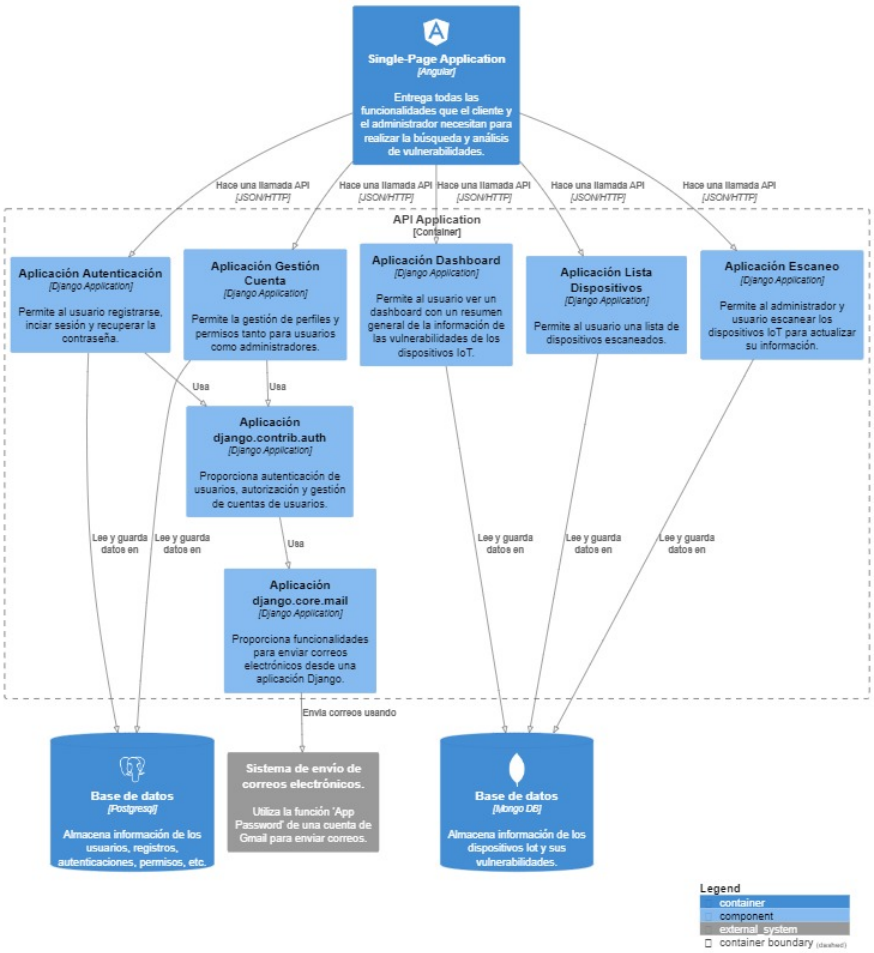


Figura 2.3: Diagrama de componentes

Las aplicaciones en Django son una forma de modularidad con el fin de separar diferentes funcionalidades, cada una de estas aplicaciones por lo general están formadas por modelos, vistas y templates, explicados anteriormente, en el diagrama podemos apreciar 5 de estas aplicaciones, Autenticación, Gestión de Cuenta, Dashboard, Lista Dispositivos y Escaneo, cada una de estas aplicaciones con una funcionalidad diferente, detallada textualmente en el diagrama. Algunas de estas aplicaciones como podemos ver, interactuan con otros módulos de Django, para el uso de modelos ya existentes como User o Group, o para el uso de la funcionalidad de envío de correos electrónicos, también podemos apreciar que algunas aplicaciones hacen uso de la base de datos Postgresql y otras hacen uso de la base de datos MongoDB.

2.2.4 Diagrama de código

El diagrama de código cuenta con la perspectiva más particular del sistema, detallando cada uno de los módulos o componentes anteriores, por lo general, en un diagrama de clases, como se puede observar en la Figura 2.4, el diagrama de clases representa la estructura del código, proporcionado a los programadores una guía al momento de codificar.

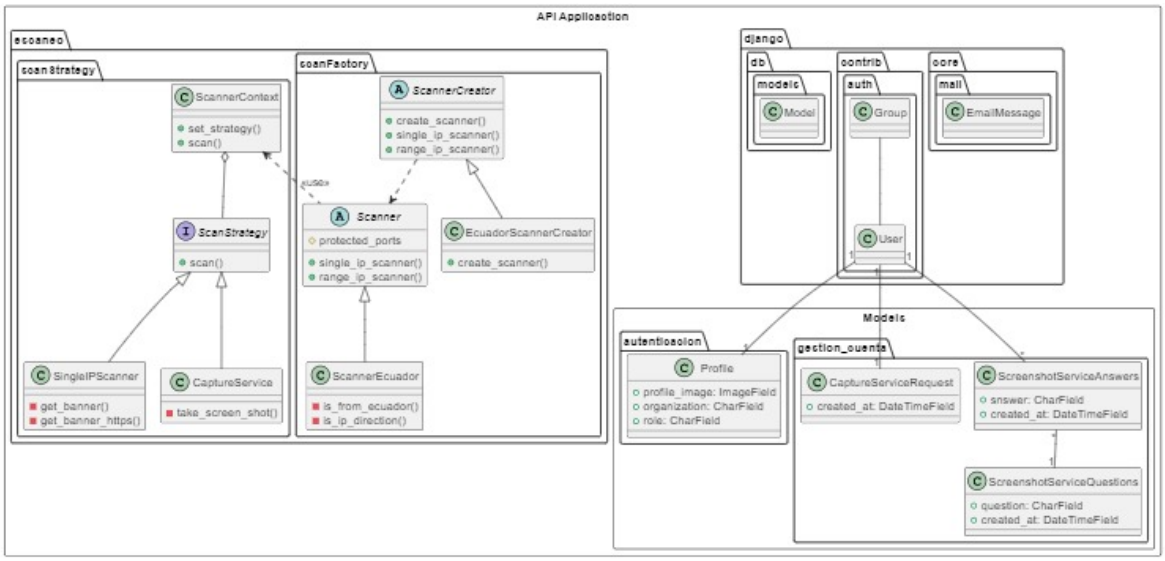


Figura 2.4: Diagrama de código

En el diagrama podemos apreciar como cada componente se detalla a través de clases, estas clases pueden ser modelos, es decir que heredan de la clase Model de Django para ser mapeadas en la base de datos, o clases ordinarias. El componente escaneo contiene la lógica de escaneo con los patrones de diseño Factory Method, para la creación de diferentes

escáneres para diferentes países, y el patrón de diseño Strategy separa los algoritmos de escaneo de puertos y la captura del servicio web, se hace uso de los patrones de diseño con el fin de que el sistema sea escalable.

Los componentes autenticación y gestión cuenta contienen clases que heredan de la clase Model de Django, en autenticación se tiene la clase Profile que contendrá información adicional de la que viene por defecto en el modelo User, como el rol, la organización y la imagen de perfil del usuario, en el componente gestión cuenta se tiene las clases para la lógica de la petición del permiso de realizar capturas de pantalla de los servicios web, como el modelo que contendrá la petición y los modelos para las preguntas y respuestas de la petición.

Adicionalmente, se representa en el diagrama algunas de las clases que vienen por defecto en Django, como los modelos User y Group, la clase Model y por último la clase EmailMessage que contiene la funcionalidad para el envío de correos electrónicos.

2.3 MOCKUPS

En el desarrollo del sistema se realizaron mockups o prototipos de las interfaces del sistema en la herramienta Figma, ANEXO VI, pero esto se realizó en la parte del componente front end, aunque resultó ser útil para obtener los servicios que son necesarios crear desde la parte del back end, así entonces se analizó cada uno de los Frames y se pensó en cada uno de los servicios que deberían tener.

2.4 HISTORIAS DE USUARIO

Los requerimientos para el sistema se obtuvieron a través de reuniones con el Product Owner, Dr. Richard Rivera, estás reuniones involucraban la discusión de las funcionalidades que debería tener el sistema, debatiendo posibilidades de agregar, quitar o modificar algunos requerimientos sugeridos para el sistema, con el fin de tener un entendimiento más amplio de lo que se desea en el sistema.

Una vez obtenidos los requerimientos, se los refinó en historias de usuario ANEXO II, agregando un nombre, descripción, rango de prioridad, etc., también se agregó criterios de aceptación con el fin de validar y verificar cada una de las historias de usuario. Para cada

historia de usuario se agregaron tareas tanto para el componente front end como el componente backend, en estas tareas se describe la historia de usuario a la que pertenece, su nombre, el tiempo estimado, el responsable de la tarea y una pequeña descripción.

2.5 PRODUCT BACKLOG

El Product Backlog, como se mencionó anteriormente, contiene los requerimientos de usuario, por lo que contiene los requerimientos refinados, es decir las historias de usuario, en cada una de estas historias se detalla el número del Sprint donde se desarrollará la historia de usuario, se detalla también el estado y la prioridad que tienen, ANEXO I. La asignación de prioridades para cada una de las historias de usuario se la realizó con la técnica Planning Poker la cual consiste en que cada miembro del equipo piense y escriba en una carta la prioridad que cree que debe tener la historia de usuario en cuestión, para después revelar cada respuesta, discutir y llegar a un consenso de la prioridad que debe tener.

2.6 SPRINT BACKLOG

El Sprint Backlog contiene las tareas creadas para cada historia de usuario ordenándolas por sprints, en el sprint backlog se clasifica estas tareas en los estados de por hacer, en progreso y hecho, con el fin de tener un control del progreso hecho en el sistema. Jira Software fue la herramienta que se utilizó para poder representar este Sprint Backlog, donde cada tarea era asignada a un miembro del equipo y cada miembro cambiaba estos estados dependiendo de su progreso.

2.7 SPRINTS

En esta sección se describirán cada uno de los sprints trabajados a lo largo del proyecto, en total se decidió dividir el trabajo en 5 sprints, detallando en esta sección también el sprint 0.

2.7.1 Sprint 0

El Sprint 0 es una fase que se realiza anterior al inicio de Scrum, en esta fase por lo general se realiza la preparación del ambiente de trabajo, estableciendo una base sólida para comenzar con la construcción del sistema, así entonces, en esta fase se realiza la preparación y configuración de todas las herramientas necesarias para la construcción del sistema.

Objetivos del Sprint

1. Preparar el ambiente de trabajo para tener una base sólida y empezar con la ejecución de los Sprint.

2. Buscar, analizar y configurar las herramientas necesarias para el desarrollo del sistema.

Preparación del repositorio

Primero se realiza la preparación del repositorio, ANEXO IV, esto con el fin de tener el código fuente centralizado en un solo lugar, y poder gestionar los cambios hechos dentro de este, teniendo cada programador el código siempre actualizado con los cambios hechos por cada uno. A continuación se describe los pasos realizados en la preparación del repositorio.

1. Se crea un repositorio privado en la página web de Github y se realizan las configuraciones necesarias.

2. Se clona el repositorio en el computador de cada uno de los programadores.

3. Se crean tres ramas, a parte de la rama principal, para el desarrollo del back end, para el desarrollo del front end y una rama para unir los cambios hechos en cada uno y verificar que no exista conflictos entre ambos.

4. Finalmente se modifica el archivo README con la descripción del proyecto y se crean dos carpetas para el componentes back end y front end como se ve en la Figura 2.5.

Configuración de las bases de datos

La configuración de las bases de datos consiste en la creación y preparación de cada una de las bases de datos, Postgresql y MongoDB, en este caso, se optó por levantar las bases de datos en la nube para tener los datos en un solo lugar y no de forma local en cada una de las computadoras, facilitando tener un acceso y modificación global de los datos. Los servicios para levantar o hostear las bases de datos son Tembo para la base de datos Postgresql y MongoDB Atlas para la base de datos MongoDB.

Tembo es un sitio web que permite hostear bases de datos postgresql en la nube, existen algunos planes del servicio, incluyendo del plan gratuito del que se hizo uso, este plan proporciona 10 GB de almacenamiento, 0.25 vCPU y 1GB de memoria, este servicio también nos proporciona un dashboard con la información en tiempo real del estado de la base de datos, como se muestra en la Figura 2.6.

Figura 2.5: Preparación del repositorio

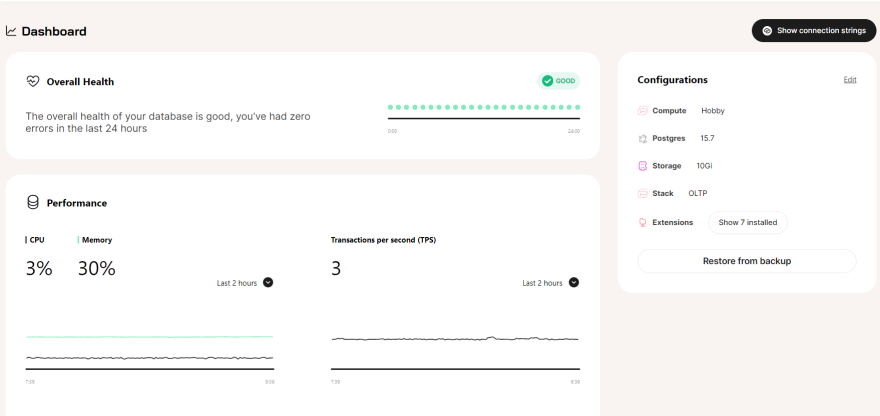


Figura 2.6: Tembo

Igualmente MongoDB Atlas es un sitio web para desplegar bases de datos MongoDB, a diferencia de Tembo este servicio solo permite levantar una base de datos gratuita, y con recursos más limitados, 5GB de almacenamiento, memoria y cpu compartidos, también ofrece un panel de administración para la configuración de la base de datos como se puede ver en la Figura 2.7.

Configuración de Django

En la configuración de Django se realiza la creación del proyecto de Django y su preparación, esto con el fin de sentar las bases para iniciar la construcción del sistema, a continuación se detallan las actividades realizadas en esta parte.

1. Primero se crea el proyecto Django dentro del repositorio creado anteriormente.

2. Se crean las aplicaciones de Django detalladas en el diagrama de componentes y se las registra en el archivo settings.py de Django.

3. Se crea un archivo local_settings.py para guardar las credenciales de las bases de datos.

4. Se crea el archivo .gitignore para ignorar algunos archivos para los cambios realizados en el repositorio, incluyendo el archivo mencionado en el punto 3.

5. Por último se realizan las migraciones iniciales a la base de datos.

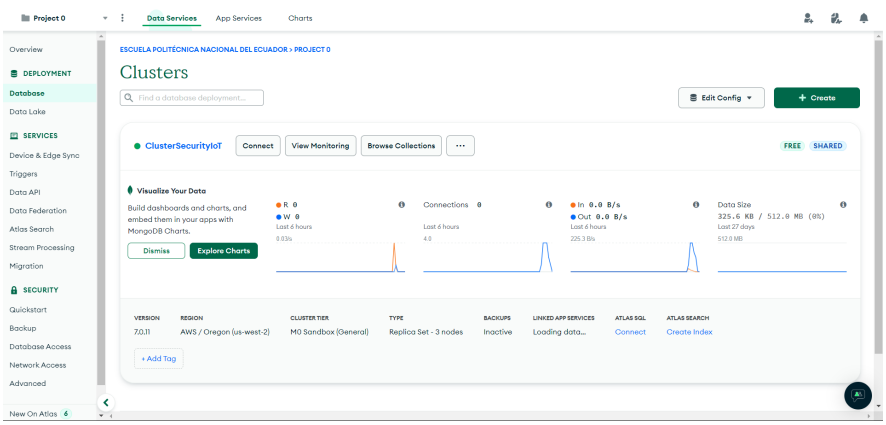


Figura 2.7: MongoDB Atlas

2.7.2 Sprint 1

Objetivos del Sprint

1. Buscar e implementar una herramienta útil para el escaneo de puertos en los dispositivos IoT.
2. Empezar con la creación del patrón de diseño detallado en el diagrama de código.
3. Implementar el módulo simple JWT para la seguridad en las APIs.
4. Buscar un método eficiente para escanear los rangos de las direcciones IP.

Análisis

En el sprint 1 se desarrolla los requisitos considerados los más importantes, es decir, los que aportan un mayor valor al producto, las historias de usuario a desarrollar son las historias de usuario con ID HU005 y HU010, ambas con una prioridad alta, en la Tabla 2.4 se muestran las historias de usuario a implementar de una manera más detallada.

Tabla 2.4: Requisitos para el Sprint 1

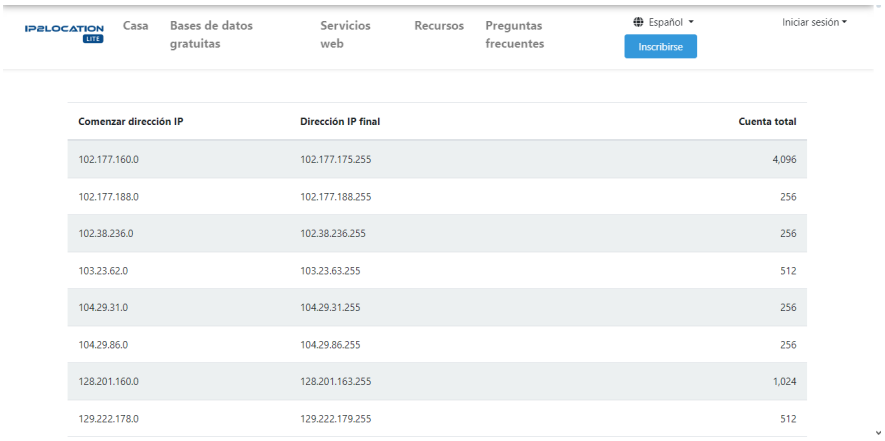
ID-HU	Nombre	Descripción	Prioridad
HU005	Escanear completa o parcialmente las direcciones IP	Como administrador, deseo contar con la capacidad de realizar escaneos completos o parciales de las direcciones IP dentro del territorio ecuatoriano para actualizar la información sobre el rango de direcciones IP asociadas a los dispositivos IoT identificados.	Alta
HU0010	Mostrar información detallada de vulnerabilidades.	Como usuario quiero obtener información detallada al buscar una dirección IP, para conocer más a fondo los detalles del escaneo realizado en cada dispositivo.	Alta

La historia de usuario HU005 tiene que ver con el escaneo de los puertos de las direcciones IP y ver cual se encuentra vulnerable, es decir, cual puerto se encuentra abierto, por lo que

se debe crear el escáner, tal como se muestra en el diagrama C4 de código, crear él o los servicios para realizar este escaneo de rangos parciales o completos de las direcciones IP en el territorio ecuatoriano y, por último, guardar la información obtenida en la base de datos MongoDB, que es la base de datos que debe contener la información relacionada a los dispositivos IoT.

Así entonces, para la creación del escáner, se debe tener en consideración dos aspectos, con qué herramienta o librería de python realizar el escaneo de puertos y qué puertos escanear, para encontrar una herramienta de escaneo de puertos se realizó una investigación donde se descubrió la librería socket de python, esta librería ofrece una forma de comunicación entre procesos IPC (Inter-Process Communication o Comunicación entre Procesos) para enviar mensajes a través de la red [22], permitiendo conocer si un puerto se encuentra abierto o no al momento de realizar esta comunicación, el criterio de selección de los puertos a escanear se lo hizo investigando y seleccionando los puertos abiertos más comunes sumando algunos puertos utilizados en varios dispositivos IoT.

Para los servicios API Restful que se deben implementar, se analizaron los mockups, donde se optó por el uso de una tabla que contiene una lista de todos los rangos de direcciones IP dentro del territorio ecuatoriano obtenido en la página web IP2Location como se ve en la Figura 2.8, por lo que para la historia de usuario HU005 deben existir dos servicios, el servicio que devuelva esta lista de rangos y otro servicio que escanee el rango seleccionado.



Comenzar dirección IP	Dirección IP final	Cuenta total
102.177.160.0	102.177.175.255	4,096
102.177.188.0	102.177.188.255	256
102.38.236.0	102.38.236.255	256
103.23.62.0	103.23.63.255	512
104.29.31.0	104.29.31.255	256
104.29.86.0	104.29.86.255	256
128.201.160.0	128.201.163.255	1,024
129.222.178.0	129.222.179.255	512

Figura 2.8: Lista de rangos de direcciones IP del sitio web IP2Location Lite

Para la historia de usuario HU010 se debe crear un servicio que devuelva la información de un dispositivo obtenido en la base de datos de la colección que contiene los dispositivos escaneados, en el caso de que un dispositivo no se encuentre en la base de datos entonces se debe escanear únicamente este dispositivo y obtener la información del dispositivo recién

escaneado.

Además, en la información de un dispositivo IoT se decidió incluir información de cada uno de los puertos listados para su análisis, por lo que se deberá incluir un servicio que devuelva información con la descripción, el protocolo y las posibles vulnerabilidades de cada puerto.

La base de datos responsable de guardar esta información es la base de datos no relacional MongoDB, por lo que se hará uso de la librería pymongo de python para gestionar la conexión a la base de datos.

Desarrollo

Comenzando con la historia de usuario HU005 primero se crea el escáner, por lo que se sigue el diseño creado en el diagrama de código, el patrón de diseño Factory Method será el responsable de crear y devolver el escáner necesario, en este caso solo el escáner para el territorio ecuatoriano, ScannerEcuador, el cuál realiza la restricción de permitir el escaneo a únicamente direcciones IP pertenecientes al territorio ecuatoriano y contendrá los puertos que se desean escanear en un lista.

Los escáneres creados y devueltos desde este patrón deben tener entonces dos métodos, uno para escanear una única dirección IP y otro para escanear uno de los rangos de la lista de rangos explicados anteriormente, el escáner entonces guarda esta información de los puertos abiertos en la base de datos más la información importante de las direcciones IP, como el país, la provincia o la ciudad, obteniendo esta información con el uso de la librería geoip2. Dentro de este patrón se hará uso del otro patrón de diseño Strategy el cuál contiene dos algoritmos, el escaneo de puertos abiertos y la captura de pantalla del posible servicio web.

Para esta historia de usuario, se desarrolla únicamente el algoritmo de escaneo de puertos, este algoritmo debe devolver un diccionario con el puerto y el banner obtenido como respuesta, en el caso de que el puerto este abierto.

El primer servicio de la historia de usuario HU005 debe devolver como respuesta los rangos de las direcciones IP del Ecuador, para esto se descarga esta información desde la página IP2Location Lite, se la convierte en formato csv y se coloca en la base de datos como una colección desde el gestor de bases de datos MongoDB Compass, como se puede ver en la Figura 2.9, así entonces el servicio da como respuesta los documentos de esta colección

de rangos pertenecientes a Ecuador.


```
_id: ObjectId('66315519d71d24e5336915a2')
id : 4
country : "Ecuador"
last_update_date : 2024-04-26T20:19:44.581+00:00
initial_ip : "8.242.88.0"
final_ip : "8.242.91.255"
num_ip : 1024
province : "Pichincha"
```


Figura 2.9: Ejemplo de un documento de la colección con los rangos de las direcciones IP de Ecuador.

El segundo servicio debe escanear el rango de direcciones IP seleccionada, por lo que como para parámetro se debe enviar el id del documento de la colección, crear el escáner utilizando el patrón de diseño y al realizar la petición al servicio se escaneará el rango de direcciones IP, agregando en la base de datos, por cada dirección IP, un documento en la colección con la información del dispositivo o actualizarlo si ya existe como se ver en la Figura 2.10.


```
_id: ObjectId('66513cf3794dac2b17c50112')
ip_address : "190.15.133.21"
status : "Activo"
created : 2024-05-24T20:20:51.801+00:00
updated : 2024-06-24T20:42:10.806+00:00
city : "Quito"
country : "Ecuador"
province : "Pichincha"
▾ vulnerability : Array (2)
  ▾ 0: Object
      port : 443
      banner : "HTTP/1.1 200 OK
               Server: nginx
               Date: Tue, 25 Jun 2024 01:42:11 GMT
               C…"
  ▾ 1: Object
      port : 80
      banner : "HTTP/1.1 301 Moved Permanently
               Location: https://:443/
               Content-Lengt…"
```


Figura 2.10: Ejemplo de un documento de la colección con los dispositivos IoT escaneados.

El primer servicio de la historia de usuario HU0010 debe obtener y devolver como respuesta la información de un dispositivo IoT de la base de datos, lo escanea en el caso de no encontrarlo, y devuelve como respuesta esta información del dispositivo.

El segundo servicio debe devolver como respuesta la información de un puerto específico, para esto, a través de un archivo csv, se coloca información de cada uno de los puertos, el protocolo, la descripción y las posibles vulnerabilidades, y se guarda esta información

en una nueva colección en la base de datos Figura 2.11, así entonces, este servicio se encarga de devolver como respuesta la información perteneciente a un puerto pasado como parámetro.

```
_id: ObjectId('6686fccd4d15c8cbede81e52')
id : 1
port_number : 443
protocol : "TCP"
description : "Acceso seguro a sitios web, aplicaciones y servicios en línea que requ…"
vulnerabilities : "Versiones obsoletas o mal configuradas de SSL/TLS, posibles ataques co…"

_id: ObjectId('6686fccd4d15c8cbede81e53')
id : 2
port_number : 80
protocol : "TCP"
description : "Acceso a sitios web y servicios en línea que no requieren cifrado de d…"
vulnerabilities : "Datos enviados a través de HTTP no están cifrados, ataques hombre en e…"
```

Figura 2.11: Ejemplo de documentos de la colección con los puertos y su información.

Así entonces, en la base de datos MongoDB existen 3 colecciones, la colección que guarda los documentos con los rangos pertenecientes al territorio ecuatoriano, la colección donde se guarda y actualiza la información de los dispositivos IoT, y la colección con la información relevante de cada puerto.

Resultado

En el sprint 1 entonces, se crearon el escáner que se detalla en el diagrama de código, exceptuando el algoritmo de capturas de pantalla dentro del patrón de diseño Strategy, y se creó los servicios para cada historia de usuario.

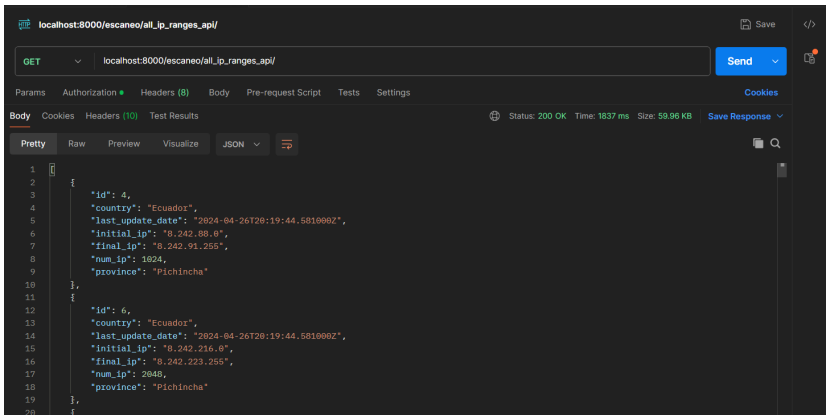


Figura 2.12: Resultado de la primera API de HU005. Obtener los rangos de direcciones IP del territorio ecuatoriano.

El primero servicio web RESTful de la historia de usuario HU005, con el método HTTP GET,

devuelve como respuesta un documento JSON con la información de todos los rangos que pertenecen al territorio ecuatoriano, como se puede ver en la Figura 2.12.

El segundo servicio web RESTful de la historia de usuario HU005, con el método HTTP POST, escanea cada uno de las direcciones IP de un rango especificado con el id del documento dado como parámetro, crea o actualiza cada uno de los dispositivos IoT, y da como resultado una respuesta confirmando el escaneo de todos las direcciones IP de ese rango.

El primer servicio web RESTful de la historia de usuario HU010, con el método HTTP GET, devuelve como respuesta la información de un dispositivo IoT en el caso de encontrarse en la base de datos, caso contrario realiza el escaneo de este dispositivo y devuelve esta información.

El segundo servicio web RESTful de la historia de usuario HU010, con el método HTTP GET, devuelve como respuesta la información de un puerto Figura 2.13, donde el número de puerto se lo pasa como parámetro.

Cabe mencionar que para cada servicio de ahora en adelante, se implementa los tokens de acceso JWT de la librería simple-jwt para consumir un servicio, incluyendo decoradores en cada una de las vistas para que cada servicio pueda ser consumido estando autenticado o teniendo los privilegios de administrador, dependiendo de cada servicio.

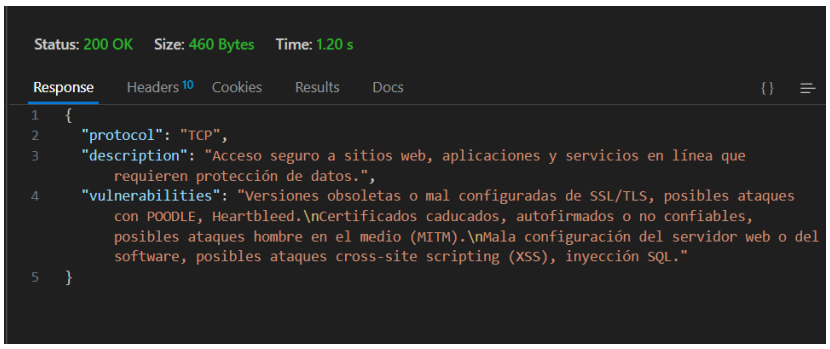


Figura 2.13: Resultado de la segunda API de HU010. Obtener la información relevante de un puerto, puerto 443.

2.7.3 Sprint 2

Objetivos del Sprint

1. Implementar un método óptimo para obtener los datos de la base de datos para los diferentes diagramas.
2. Añadir índices en las colecciones para que las consultas sean más eficientes.

Análisis

En el sprint 2 se desarrollan las 2 siguientes historias de usuario con prioridad alta, las historia de usuario con ID HU004 y HU008, ambos relacionados con la creación del dashboard dinámico para visualizar gráficamente estadísticas de las vulnerabilidades encontradas, estos requisitos se los detalla en la Tabla 2.5.

Tabla 2.5: Requisitos para el Sprint 2

ID-HU	Nombre	Descripción	Prioridad
HU004	Visualizar mapa de calor de direcciones IP y puertos.	Como usuario quiero visualizar a manera de mapa geográfico el número de vulnerabilidades existentes en cada provincia del país, para poder identificar las áreas con mayor riesgo de seguridad.	Alta
HU008	Visualizar dashboard.	Como usuario quiero visualizar estadísticas de manera gráfica que muestre el número total de dispositivos vulnerables en base a determinados parámetros, para obtener la información relevante de manera ordenada.	Alta

Los servicios que deben crearse igualmente se basaron en los mockups, por lo que analizando los frames en Figma vemos que en el dashboard se decidió añadir el mapa de calor más 3 diagramas adicionales como vemos en la Figura 2.14, donde la parte de interactividad involucra seleccionar una provincia en el mapa de Ecuador y obtener los 3 diagramas con la información de únicamente esa provincia.

Los 3 diagramas son: un diagrama que muestra el número de dispositivos vulnerables to-

tales, un diagrama pastel donde muestra el porcentaje del estado de los dispositivos y por último, un diagrama de barras que representa cada puerto y la cantidad de dispositivos que presentan abierto cada puerto. El mapa de calor únicamente muestra la cantidad de dispositivos vulnerables por provincia donde la cantidad se representa por colores.

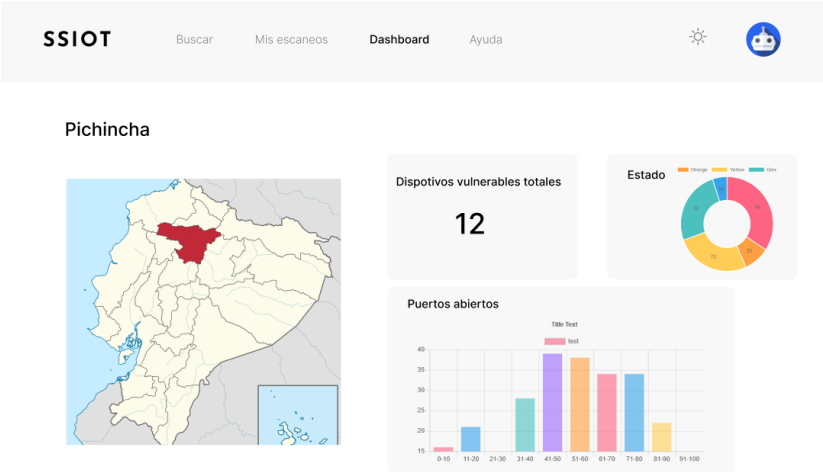


Figura 2.14: Frame del dashoboard en Figma.

Así entonces, después de analizar el dashboard bosquejado en Figma, los servicios que se deben crear son 2, una para cada historia de usuario, un servicio para el mapa de calor que devuelva los dispositivos vulnerables por provincia y un servicio que abarque los 3 diagramas, con un parámetro donde se especifique la provincia de la cual se quiere obtener la información para dibujar los diagramas.

La colección de dispositivos IoT en un principio, se espera que contenga la información de todos los dispositivos presentes en Ecuador, lo que significa que se espera que lleguen a ser millones de documentos dentro de la colección, por ende se debe optimizar estas consultas para que tener un mejor rendimiento.

Desarrollo

En el desarrollo entonces se deben crear 2 servicios que a través de consultas a la base de datos MongoDB devuelvan la información necesaria, para esto primero se construirá el primer servicio mencionado, el servicio para el mapa de calor, obteniendo los datos a través de una consulta propia de MongoDB Figura 2.15 con ayuda de la librería pymongo, así entonces, para esta consulta se realiza una agregación por provincia y se hace un conteo de los dispositivos que presenten el campo de vulnerabilidades, creando un índice en el

campo provincia para optimizar la consulta.

```
pipeline = [
    {
        "$group": {
            "_id": "$province",
            "vulnerables": {"$sum": {"$cond": [{"$gt": [{"$size": { "$ifNull": [ "$vulnerability", [] ] }}, 0]}, 1, 0]}},
        }
    },
    {
        "$project": {
            "provincia": "$_id",
            "vulnerables": 1,
            "_id": 0
        }
    },
    {
        "$replaceRoot": {
        "newRoot": {
        "$arrayToObject": [[{ "k": "$provincia", "v": "$vulnerables" }]]
        }
        }
    }
]

results = collection.aggregate(pipeline)
```

Figura 2.15: Consulta para obtener la información necesaria del mapa de calor.

El segundo servicio con respecto a los diagramas, también se realizará a través de consultas MongoDB, pero en este caso se realizarán 3 consultas separadas en 3 funciones para después unirlas en un solo servicio, además estas consultas deben recibir como parámetro la provincia para obtener la información únicamente de los dispositivos de esa provincia.

Resultado

Así entonces, el servicio web RESTful de la historia de usuario HU004, con el método HTTP GET, retorna un JSON con el documento incluyendo dentro de este las claves (provincias) y los valores (número de dispositivos vulnerables), esto servirá para dibujar el mapa de calor en el front end, así entonces, a través de una consulta se irán obteniendo cada una de las provincias que se encuentran en la base de datos y se realiza un conteo de los dispositivos que tienen vulnerabilidades, es decir, los documentos que tienen el campo vulnerabilidades.

El servicio web RESTful de la historia de usuario HU008, con el método HTTP GET, retorna un JSON con el documento que contiene 3 documentos internos con la información para los otros 3 diagramas, el primer documento contiene únicamente el número de dispositivos vulnerables, el segundo contiene los estados de los dispositivos y el tercero contiene los puertos abiertos Figura 2.16, cada uno para los 3 diagramas, pasando si se desea como parámetro la provincia de la cuál se quiere obtener esta información, de no ser el caso esta información será con respecto a todo el territorio ecuatoriano.

Status: 200 OK Size: 150 Bytes Time: 1.62 s

Response Headers 10 Cookies Results Docs { }


```
1  {
2    "vulnerables": 20,
3    "status": {
4      "Activo": 17,
5      "Inactivo": 3
6    },
7    "ports": {
8      "21": 1,
9      "80": 9,
10     "443": 20,
11     "554": 1,
12     "1024": 1,
13     "2000": 1,
14     "5060": 1,
15     "8080": 2,
16     "8443": 1,
17     "10000": 1
18   }
19 }
```


Figura 2.16: Resultado de la API de HU008. Servicio para obtener la información de los diagramas del dashboard.

2.7.4 Sprint 3

Objetivos del Sprint

1. Implementar el escáner para realizar el re escaneo de un dispositivo.
2. Buscar una solución para registrar el historial de los escaneos hechos a un dispositivo.

Análisis

En el sprint 3 se desarrollan los requisitos posteriores con prioridad media, las historias de usuario con el ID HU006 y HU007, la primera se relaciona con el re escaneo de un dispositivo IoT y la segunda se relaciona con el historial o registro de todos los escaneos de un dispositivo, estas historias de usuario se detallan en la Tabla 2.6.

Tabla 2.6: Requisitos para el Sprint 3

ID-HU	Nombre	Descripción	Prioridad
HU006	Re escaneo de un dispositivo.	Como usuario quiero actualizar la información de las vulnerabilidades de dispositivos IoT mediante un re escaneo, para conocer si las vulnerabilidades se han mitigado, han aumentado o siguen igual.	Media
HU007	Mostrar historial de escaneos.	Como usuario quiero visualizar todos los escaneos anteriores que se realizaron a un dispositivo IoT para poder conocer los cambios de estado o vulnerabilidades que pueden existir en cada escaneo.	Media

El servicio para la historia de usuario HU006 debe ser únicamente el escaneo de un dispositivo IoT pasando como parámetro la dirección IP y devolviendo la información obtenida del escaneo.

Para la historia de usuario HU007 se debe agregar en la base de datos cada uno de los escaneos hechos a un dispositivo, por lo tanto se debe modificar el proceso de escaneo para que cada vez que se ejecute guarde en una nueva colección todos los escaneos hechos a una dirección IP, por lo tanto, únicamente se debe crear un servicio para obtener todos los escaneos hechos a una dirección IP.

Desarrollo

El servicio de la historia de usuario HU006 simplemente ocupa el patrón de diseño Factory Method para crear y obtener el escáner, después ocupa el método que escanea una única dirección IP, dada como parámetro, obtiene esta información de la base de datos y por último devuelve esta información como respuesta.

Para la historia de usuario HU007, entonces se crea una nueva colección para agregar cada uno de los escaneos hechos a un dispositivo, en el escaneo se añade el algoritmo para agregar un dispositivo a la colección cada vez que se ejecuta, y por último se crea el servicio para devolver todos los escaneos hechos a un dispositivo IoT, pasando como parámetro la dirección IP del dispositivo.

Por lo tanto, en la base de datos MongoDB existe otra nueva colección que contendrá todo el historial de los escaneos hechos a las direcciones IP, a parte de la colección que contiene los dispositivos actualizados con su último escaneo.

Resultado

Así entonces, el servicio web RESTful, con el método HTTP GET para la historia de usuario HU006, escanea una dirección IP dada como parámetro con el uso del escáner obtenido del patrón de diseño Factory Method, por lo que este servicio da como respuesta la información del dispositivo IoT escaneado Figura 2.17.

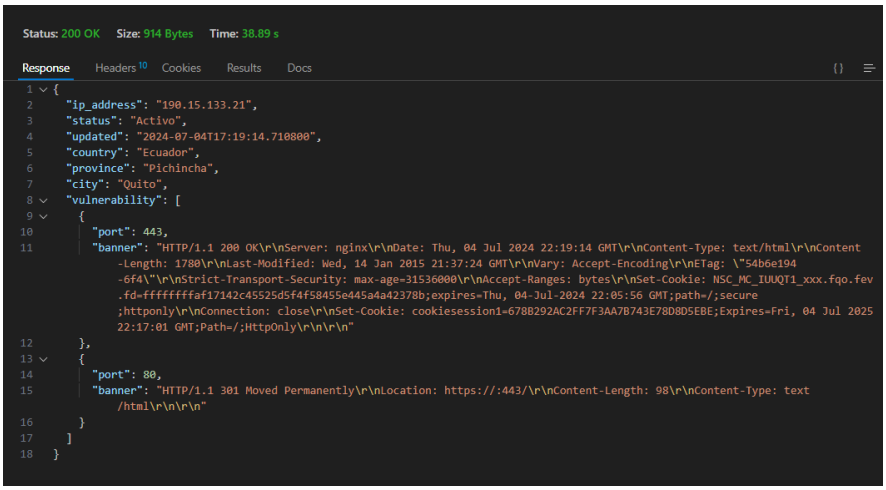


Figura 2.17: Resultado de la API del HU006. Servicio para re escanear una dirección IP.

Por último, el servicio web RESTful, con el método HTTP GET para la historia de usuario HU007, devuelve como respuesta todos los escaneos realizados a un dispositivo IoT, donde se tiene que pasar como parámetro la dirección IP de este dispositivo.

2.7.5 Sprint 4

Objetivos del Sprint

1. Implementar buenas medidas de seguridad para el registro e inicio de sesión.

2. Buscar e implementar una herramienta eficiente para realizar la captura de los servicios web.

3. Crear la función de capturas de pantalla restante en el patrón de diseño.

Análisis

Las historias de usuario para el sprint 4 son 2, ambas con prioridad media, la historia de usuario con ID HU001 y HU009, la primera se relaciona con el proceso de autenticación del usuario y la segunda se relaciona con la captura de los servicio web tal como se detalla en la Tabla 2.7, cabe mencionar que en este sprint también se creará la lógica del permiso del usuario para realizar la captura de pantalla, creando un nuevo grupo con el Model Group de Django, con el fin de que el servicio de captura sea únicamente accesible para los usuario con este permiso.

Tabla 2.7: Requisitos para el Sprint 4

ID-HU	Nombre	Descripción	Prioridad
HU001	Registrar e iniciar de sesión.	Como administrador deseo que el usuario se registre antes de ingresar al sistema para validar el acceso de cada cuenta y tener un registro de las acciones realizadas dentro del sistema.	Media
HU009	Mostrar captura de servicios web.	Como usuario deseo tener una captura del servicio web correspondiente a cada dispositivo IoT, de ser el caso, para conocer el tipo de servicio que se encuentra vulnerable.	Media

En la historia de usuario HU001 se debe realizar todo el proceso de registro, inicio de sesión y recuperar contraseña, por lo que se debe crear el servicio de registro y un servicio para iniciar sesión, el proceso recuperar la contraseña no se la realiza como servicio API Rest si no que utiliza las vistas propias de Django que implementan ya este proceso.

Para la historia de usuario HU009 se debe implementar el algoritmo pendiente dentro del patrón de diseño Strategy para la captura de pantalla de los posibles servicios web, y utilizar este algoritmo en el servicio para generar y obtener la captura de pantalla.

Desarrollo

De la historia de usuario HU001, el primer servicio que se construye es el proceso de registro, en este servicio entonces se dan como parámetros las credenciales y la información adicional del usuario, correo electrónico, nombre, empresa y rol, adicionalmente se decidió agregar una imagen de perfil automática para cada usuario, después de esto el servicio envía un correo electrónico con un enlace para la verificación del correo electrónico registrado Figura 2.18, el enlace entonces redirige a la vista que se encarga de activar la cuenta de usuario, con lo que el usuario ya puede iniciar sesión.

El servicio de iniciar sesión de la historia de usuario será el servicio que ofrece por defecto la librería simple jwt para obtener los tokens de acceso con el usuario y su contraseña, por lo que simplemente se añaden estas vistas a la aplicación.

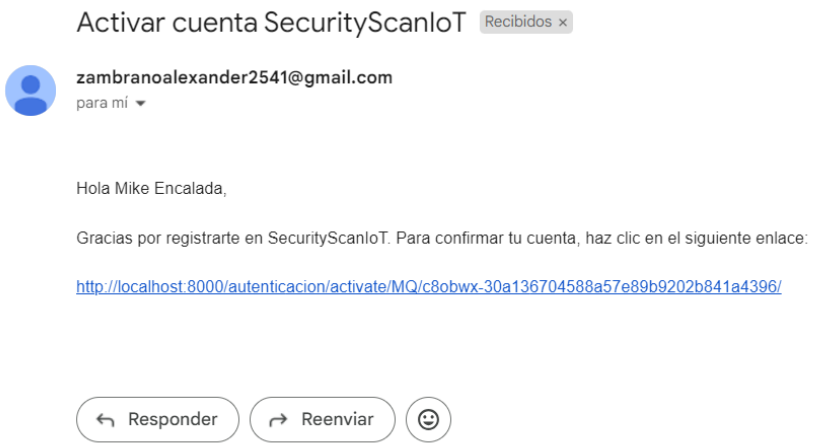


Figura 2.18: Ejemplo del correo electrónico enviado para la confirmación y activación de la cuenta de usuario.

Por último para esta historia de usuario, se implementan las vistas que vienen por defecto en Django para la recuperación de contraseñas y tan solo se modifican los templates o plantillas para que sea consistente al diseño de la página web.

El servicio para la historia de usuario HU009, con el uso de la librería pyppeteer, genera la captura del servicio web, después se guarda en el fichero media donde se guardan las imágenes en Django, y por último se genera y se devuelve el url para el acceso a esta imagen, para el consumo de este servicio entonces se proporciona una dirección IP y el puerto, en este caso únicamente los puertos 80, 8080 o 443.

Resultado

Así entonces, para la historia de usuario HU001 se crearon 2 servicios para registro e inicio de sesión, el primer servicio web RESTful, con el método HTTP POST, registra al usuario, enviando un correo electrónico para la confirmación y activación de la cuenta.

El segundo servicio web RESTful, con el método HTTP POST, da como respuesta el token de acceso y el token para actualizarlo Figura 2.19, una vez que la autenticación sea válida y la cuenta esté activa. Cabe mencionar que el cifrado de contraseña lo hace directamente el Framework de Django por lo que no se tuvo que implementar ningún algoritmo de cifrado.

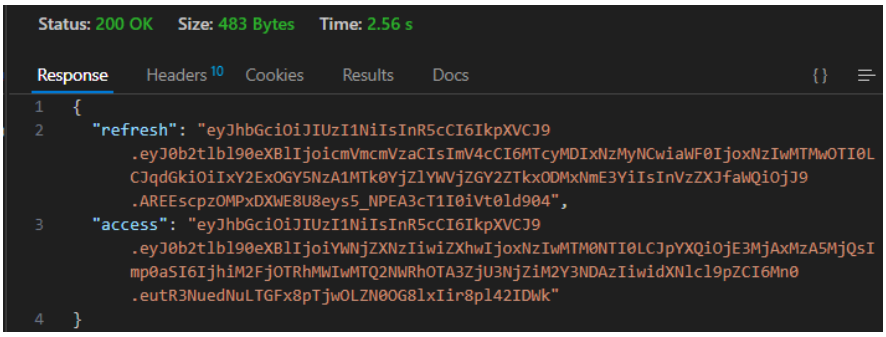


Figura 2.19: Resultado de una de las API del HU001. Servicio para iniciar sesión y obtener los tokens de acceso.

El servicio web RESTful de la historia de usuario HU009, con el método HTTP GET, genera la imagen de la captura del servicio web, si existe, y devuelve como respuesta la url de la imagen generada Figura 2.20.

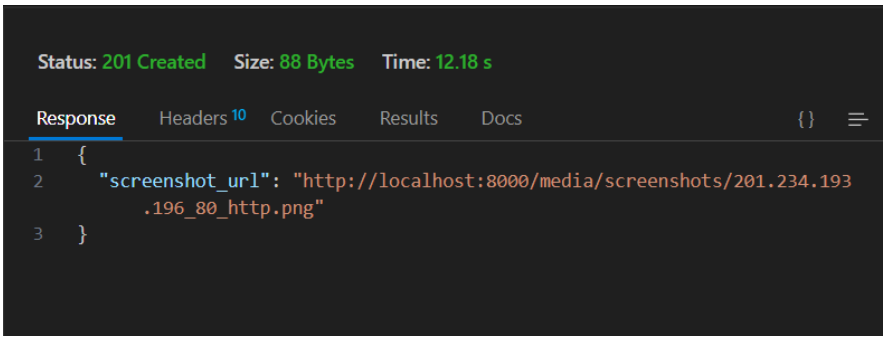


Figura 2.20: Resultado de la API del HU009. Servicio para generar y obtener la imagen de la captura de pantalla de un servicio web.

Para la historia de usuario HU009, también se crearon los servicios para la gestión de permisos del usuario para visualizar la captura de pantalla, así entonces, el segundo servicio web RESTful de la historia de usuario HU009, con el método HTTP POST, permite al usuario realizar está petición de permiso a través de un formulario.

Por último, otros servicios web RESTful de la historia de usuario HU009, permiten al administrador obtener estás solicitudes con las respuestas del usuario del formulario, este formulario consta de 2 preguntas de seguridad que el usuario tiene que contestar para que el administrador decida, con el uso de otro servicio, aceptar o eliminar la solicitud, al eliminar la solicitud, únicamente se borra esta solicitud de la base de datos, y si lo acepta entonces se añade el usuario al grupo de los usuarios que tienen este permiso.

2.7.6 Sprint 5

Objetivos del Sprint

1. Buscar una solución para almacenar la información de los escaneos que desean guardar de los usuarios.

2. Refactorizar el código si es necesario.

Análisis

En el sprint 5 finalmente se desarrolla las últimas dos historias de usuario con un prioridad baja, la historia de usuario con ID HU002 y HU003, la primera historia de usuario se relaciona con el perfil del usuario y la segunda se relaciona con la gestión de cuentas realizado por el administrador, como se ve en la Tabla 2.8.

Tabla 2.8: Requisitos para el Sprint 5

ID-HU	Nombre	Descripción	Prioridad
HU002	Visualizar mi perfil.	Como usuario quiero visualizar los datos de mi perfil y escaneos realizados para agilizar el proceso de revisar y gestionar mis búsquedas, brindándome una experiencia más rápida y personalizada.	Baja
HU003	Gestionar cuentas.	Como administrador deseo que las cuentas del usuario puedan gestionarse para tener control de estas, logrando así, activar y desactivar cada una de las cuentas.	Baja

En la historia de usuario HU002 entonces se debe crear el servicio para obtener la información del perfil del usuario, y otro servicio para editar esta información si es posible, incluyendo la actualización o cambio de contraseña por parte del usuario.

Adicionalmente la historia de HU002 indica que debe existir una funcionalidad para que el usuario pueda guardar los escaneos deseados, con el fin de tener un registro de los escaneos y poder acceder a estos sin la necesidad de buscar nuevamente un dispositivo, por lo que es necesario crear dos servicios para esta funcionalidad.

Los servicios a construir para la historia de usuario HU003 deben ser, un servicio para obtener a todos los usuarios y un servicio para activar o desactivar una cuenta de usuario, al desactivar una cuenta de usuario esta no podrá iniciar sesión ni obtener el token para consumir los servicios.

Desarrollo

La historia de usuario HU002 tiene 3 servicios, el primer servicio devuelve la información de perfil del usuario actual, el segundo servicio actualiza la información del perfil de usuario, el primer nombre, segundo nombre, la institución y el rol, y el tercer servicio actualiza la contraseña del usuario, enviando su contraseña actual para validar este cambio.

Para la historia de usuario HU003, se crean dos servicios uno que devuelva la lista de todos los usuarios y otro servicio que desactive o active una cuenta de usuario, por lo que en el primer servicio simplemente se hace una consulta utilizando los métodos de clase Model User de Django para obtener todos los usuarios, y para el segundo servicio igualmente solo se actualiza el campo is_active de este modelo, en este servicio se debe pasar como parámetro el id del usuario.

Resultado

El primer servicio web RESTful de la historia HU002, con el método HTTP GET, entonces devuelve la información del perfil del usuario, como el nombre de usuario, primer nombre, segundo nombre, correo, etc. El segundo servicio web RESTful, con el método HTTP PUT, actualiza la información del perfil del usuario, y el tercer servicio web RESTful, con el método

HTTP PUT, cambia la contraseña del usuario pasando como cuerpo la contraseña actual del usuario.

En la historia de usuario HU003, el primer servicio servicio web RESTful, con el método HTTP GET, devuelve todos la información de todos los usuarios registrados en el sistema Figura 2.21, incluyendo si la cuenta de usuario está activa y al grupo al que pertenece, el segundo servicio web RESTful, con el método HTTP PUT, activa o desactiva la cuenta de un usuario, pasándole como parámetro un valor booleano.

Además, para la historia de usuario HU002, se crearon dos servicios adicionales, el primer servicio permite al usuario guardar un escaneo deseado, por lo que en una nueva colección se guardan cada uno de estos escaneos junto con un campo que indique el usuario al que pertenece este escaneo, y finalmente el último servicio da como respuesta todos los escaneos guardados por un usuario, para así listarlos en una sección llamada mis escaneos.

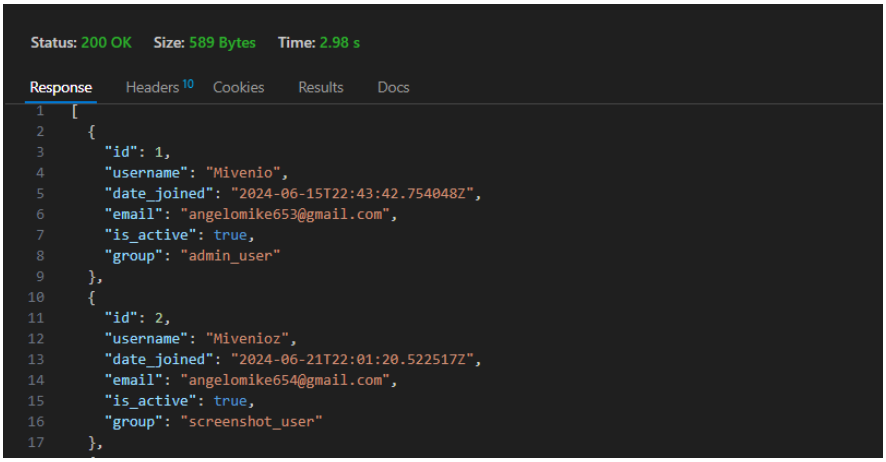


Figura 2.21: Resultado de una de las API del HU003. Servicio para obtener todos los usuarios registrados.

Por último, se cambiaron los nombres de algunas clases y métodos para que sea más legible el código, además se intentó modularizar el código a través de métodos para no tener código repetido, logrando así refactorizar el código.

2.8 ENTREGABLE FINAL

Al final de la ejecución de los sprints, en el componente del back-end, se completó la creación de los servicios API RESTful, listos para su integración final con el front-end, culminando así el desarrollo del sistema web. Así entonces, el entregable final es el sistema web completo y funcional de búsqueda y análisis de seguridad de dispositivos IoT.

3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES

3.1 RESULTADOS

Finalmente, al finalizar el componente back end se terminaron de crear los servicios API RESTful correspondientes para cada historia de usuario, para su integración final con el front end, el catálogo completo de estos servicios se puede ver en la Tabla 3.1.

Tabla 3.1: Catálogo de todos los servicios API RESTful creados en el compontente back end

Historia de Usuario	**Endpoint**	**Descripción**	**Método HTTP**	**ID API**
HU001	autenticacion/register/	Registra a un usuario.	POST	API1
HU001	/api/token/	Devuelve los tokens de acceso.	POST	API2
HU002	gestion_cuenta /get_my_user_info_api/	Devuelve la información de la cuenta del usuario.	GET	API3
HU002	gestion_cuenta/update_ my_profile_api/	Actualiza la información del perfil del usuario.	GET	API4
HU002	gestion_cuenta/update_my_ user_info_api/	Actualiza la información principal del usuario.	GET	API5
HU002	lista_dispositivos /get_my_scannings_api/	Devuelve la información de los escaneos de un usuario.	GET	API6
HU002	lista_dispositivos /add_a_my_scanning_api /<str:ip>/	Guarda un escaneo a mis escaneos.	POST	API7
HU002	lista_dispositivos/delete_a_my_ scanning_api/<str:ip>/	Elimina un escaneo guardado por un usuario.	DELETE	API8

HU003	gestion_cuenta /get_users_list_api/	Devuelve la lista de todos los usuarios registrados.	GET	API9
HU003	gestion_cuenta/activate_user_ api/<int:id>/	Activa o desactiva la cuenta de un usuario.	PUT	API10
HU004	dashboard /heatmap_api/	Devuelve la información para dibujar el heatmap.	GET	API11
HU005	escaneo /all_ip_ranges_api/	Devuelve la lista de todos lo rangos de direcciones IP.	POST	API12
HU005	escaneo/ip_range_scanning_api /<int:idrange>	Escanea un rango de direcciones IP específico.	POST	API13
HU006	escaneo /iot_device/<str:ip>/	Re escanea una dirección IP y devuelve esa información.	POST	API14
HU007	lista_dispositivos /get_all_scannings_api/<str:ip>/	Obtiene el historial de todos los escaneos hechos a un dispositivo.	GET	API15
HU007	lista_dispositivos /get_a_scanning_by_user_api /<int:docid>/	Obtiene el la información de un escaneo hecho a un dispositivo IoT.	GET	API16
HU008	dashboard /get_dashboard_api /<str:provincia>/	Obtiene la información para dibujar los diagramas del dashboard.	GET	API7
HU009	escaneo/screenshot/<str:ip>/ <int:port>/	Obtiene la URL para acceder la imagen de captura de pantalla.	GET	API18
HU009	gestion_cuenta/request_ screenshot_api/	Realizar una petición para el permiso de captura de pantalla.	POST	API19
HU009	gestion_cuenta /get_requests_screenshot_api/	Devuelve las peticiones de captura de pantalla hechas por los usuarios.	GET	API20
HU009	gestion_cuenta /add_to_screenshot_group_api/	Da permiso de captura de pantalla a un usuario.	POST	API21

HU009	gestion_cuenta/delete_ screenshot_request_api/ <int:usrid>/	Elimina la solicitud de permiso de captura de pantalla.	DELETE	API22
HU010	lista_dispositivos /iot_device/<str:ip>/	Devuelve la información de un dispositivo IoT.	GET	API23
HU010	lista_dispositivos /get_port_info_api/<int:port>/	Devuelve la información de un puerto.	GET	API24

Adicionalmente, en esta sección se presentarán las pruebas a las que fue sometido el componente back end una vez finalizado su desarrollo, por lo que se detallará tanto las pruebas de funcionalidad como las pruebas de rendimiento, con el fin de asegurar que cada uno de los endpoints de los servicios API RESTful funcionen correctamente, en el caso de las pruebas de funcionalidad, y verificar que el sistema se comporta y responde eficientemente en situaciones de carga de varias solicitudes, en el caso de las pruebas de rendimiento.

3.1.1 Pruebas de funcionalidad

Las pruebas de funcionalidad pondrán a prueba cada uno de los endpoints ANEXO III para asegurar que cada uno devuelva la información deseada dependiendo de diferentes parámetros, a través de diferentes casos de prueba, en este documento se detallará las pruebas hechas únicamente a los servicios más relevantes, es decir, los servicios que se relacionan con el escaneo, captura de servicio web, presentación del dashboard, registro e inicio de sesión, con el fin de no extender demasiado el documento.

En cada caso de prueba existirán varios escenarios, el escenario de éxito, en el que los datos de entrada serán correctos y el servicio responderá exitosamente, y el escenario de error, donde por el otro lado, los datos de entrada serán incorrectos y la respuesta del servicio será un mensaje de error.

Cabe mencionar que estás pruebas se realizaron con la herramienta Postman, esta herramienta permite el consumo de servicios API RESTful y también permite automatizar pruebas, indicando, a través de un script, las respuestas esperadas como el código HTML, la estructura o los datos Figura 3.1, siendo exitosa la prueba si cumple con todos resultados de prueba y, por otro lado, siendo fallida si no cumple con estos resultados.

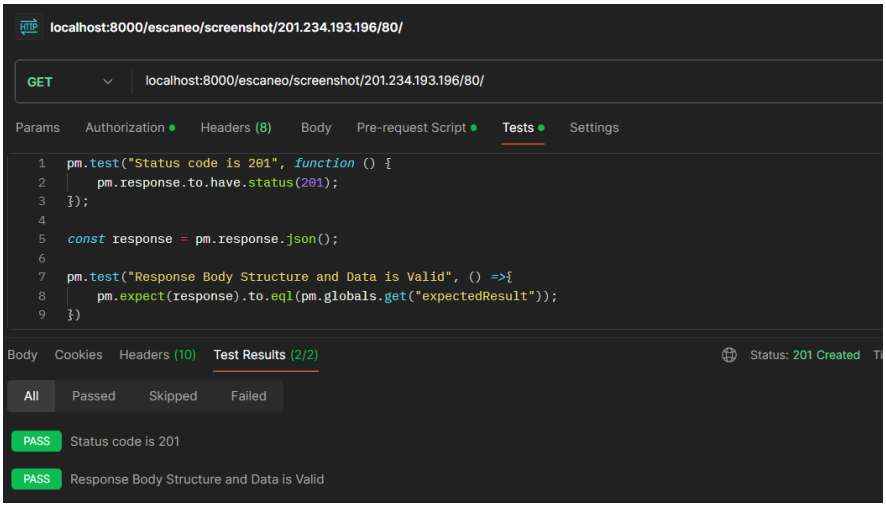


Figura 3.1: Ejemplo de prueba automatizada en Postman para el servicio API RESTful de la captura de pantalla de un servicio web.

Caso de prueba CP_API_001

En este caso de prueba se evaluará al endpoint encargado de mostrar la información detallada de vulnerabilidades, realizado por parte del usuario principal, por lo que se evaluará el servicio API RESTful con id API23 de la historia de usuario HU010.

Tabla 3.2: Resultados de la prueba de funcionalidad del servicio API RESTful con id API23 de la historia de usuario HU010

Endpoint	**Resultado Obtenido**	**Cumple Resultado**
/lista_dispositivos/iot_device /201.234.193.196/	La API devuelve un array JSON con la información del dispositivo IoT si se encuentra en la base de datos, de no encontrarse, realiza el escaneo para guardar y obtener la información. Código HTTP: 200 OK	Sí
/lista_dispositivos/iot_device /7.0.2.4.6/	La API devuelve un mensaje de error explicando que el valor de la dirección IP pasado como parámetro es inválido. Código HTTP: 400 Bad Request	Sí
/lista_dispositivos/iot_device /8.51.10.0/	La API devuelve un mensaje de error explicando que la dirección IP pasada como parámetro no pertenece a Ecuador. Código HTTP: 400 Bad Request	Sí

Caso de prueba CP_API_002

En este caso de prueba se evaluará al endpoint encargado de devolver la información relevante de los dispositivos IoT para mostrarlos de manera gráfica en un dashboard, realizado por parte del usuario principal, por lo que se evaluará el servicio API RESTful con id API7 de la historia de usuario HU008.

Tabla 3.3: Resultados de la prueba de funcionalidad del servicio API RESTful con id API7 de la historia de usuario HU008

Endpoint	**Resultado Obtenido**	**Cumple Resultado**
/dashboard/get_dashboard_api /Pichincha/	La API devuelve un array JSON con la información de los dispositivos perteneciente a una provincia, en este caso, Pichincha. Código HTTP: 200 OK	Sí
/dashboard/get_dashboard_api/	La API devuelve un array JSON con la información de los dispositivos de todo el territorio ecuatoriano. Código HTTP: 200 OK	Sí
dashboard/get_dashboard_api /EstadosUnidos/	La API devuelve un JSON con información vacía. Código HTTP: 200 OK	Sí

Caso de prueba CP_API_003

En este caso de prueba se evaluará al endpoint encargado de devolver el url para poder acceder a la imagen de la captura de pantalla del servicio web, realizado por parte del usuario principal con permisos de captura de pantalla, por lo que se evaluará el servicio API RESTful con id API18 de la historia de usuario HU009.

Tabla 3.4: Resultados de la prueba de funcionalidad del servicio API RESTful con id API18 de la historia de usuario HU009

Endpoint	**Resultado Obtenido**	**Cumple Resultado**
/escaneo/screenshot /201.234.193.196/80/	La API devuelve un array JSON con la url para acceder a la captura de pantalla del servicio web. Código HTTP: 201 Created	Sí

/escaneo/screenshot /201.234.193.196/54/	La API devuelve un JSON con un mensaje de error indicando que el puerto es incorrecto. Código HTTP: 400 Bad Request	Sí

Caso de prueba CP_API_004

En este caso de prueba se evaluará al endpoint encargado de realizar el registro de un usuario en el sistema, por lo que se evaluará el servicio API RESTful con id API1 de la historia de usuario HU001.

Tabla 3.5: Resultados de la prueba de funcionalidad del servicio API RESTful con id API1 de la historia de usuario HU001

Endpoint	**Resultado Obtenido**	**Cumple Resultado**
/autenticacion/register/	La API devuelve un array JSON indicando que se realizó el registro correctamente Código HTTP: 201 Created	Sí
/autenticacion/register/	La API devuelve un JSON con un mensaje de error indicando que ese usuario ya se encuentra registrado.	Sí

Caso de prueba CP_API_005

En este caso de prueba se evaluará al endpoint encargado de realizar el inicio de sesión de un usuario, por lo que se evaluará el servicio API RESTful con id API2 de la historia de usuario HU001.

Tabla 3.6: Resultados de la prueba de funcionalidad del servicio API RESTful con id API2 de la historia de usuario HU001

Endpoint	**Resultado Obtenido**	**Cumple Resultado**
/api/token/	La API devuelve un array JSON con el token de acceso y el token para actualizar el token de acceso	Sí

/api/token/	La API devuelve un JSON con un mensaje de error indicando que no existen cuentas activas con esas credenciales.	Sí

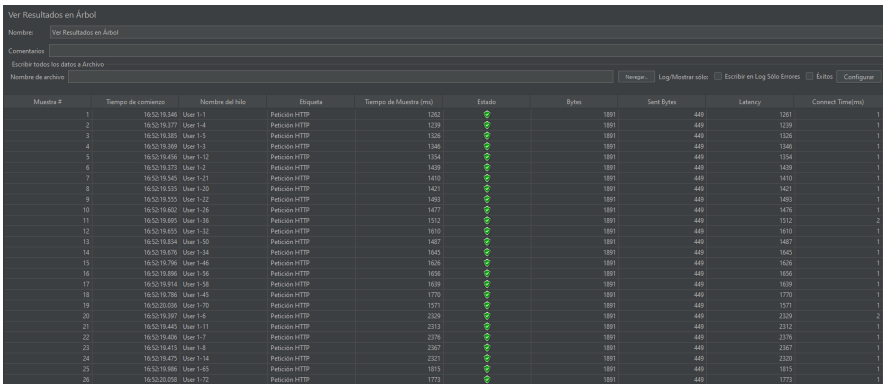
3.1.2 Pruebas de rendimiento

Las pruebas de rendimiento servirán para determinar como será el comportamiento del sistema si se encuentra bajo una gran carga de trabajo, es decir, bajo varias solicitudes hechas al sistema, en este caso se usará el servidor web que viene por defecto en Django para realizar las pruebas y determinar si el sistema se comporta eficientemente bajo ciertas condiciones o si se debería cambiar a otro servidor web, junto con otras configuraciones, para mejorar este rendimiento.

Así entonces, se hará uso de la herramienta JMeter, para pruebas de carga, verificando el rendimiento del sistema, por lo que se hará las pruebas a los servicios API RESTful más relevantes del sistema.

Prueba de rendimiento a API23

La servicio API RESTful con id API23, tiene que ver con la obtención de la información de un dispositivo IoT, algo crucial y que aporta valor al cliente, por lo que se hará la prueba de rendimiento a este servicio.



Muestra #	Tiempo de comienzo	Nombre del hilo	Etiqueta	Tiempo de Muestra (ms)	Estado	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	16:52:19.346	User 1-1	Petición HTTP	1262		1891	449	1261	1
2	16:52:19.377	User 1-4	Petición HTTP	1239		1891	449	1239	1
3	16:52:19.389	User 1-5	Petición HTTP	1528		1891	449	1526	1
4	16:52:19.389	User 1-3	Petición HTTP	1349		1891	449	1349	1
5	16:52:19.456	User 1-12	Petición HTTP	1354		1891	449	1354	1
6	16:52:19.373	User 1-2	Petición HTTP	1439		1891	449	1439	1
7	16:52:19.545	User 1-21	Petición HTTP	1410		1891	449	1410	1
8	16:52:19.535	User 1-20	Petición HTTP	1421		1891	449	1421	1
9	16:52:19.555	User 1-22	Petición HTTP	1493		1891	449	1493	1
10	16:52:19.602	User 1-26	Petición HTTP	1477		1891	449	1476	1
11	16:52:19.695	User 1-36	Petición HTTP	1512		1891	449	1512	2
12	16:52:19.655	User 1-32	Petición HTTP	1610		1891	449	1610	1
13	16:52:19.834	User 1-50	Petición HTTP	1487		1891	449	1487	1
14	16:52:19.676	User 1-34	Petición HTTP	1645		1891	449	1645	1
15	16:52:19.796	User 1-46	Petición HTTP	1626		1891	449	1626	1
16	16:52:19.896	User 1-56	Petición HTTP	1606		1891	449	1606	1
17	16:52:19.914	User 1-58	Petición HTTP	1639		1891	449	1639	1
18	16:52:19.786	User 1-45	Petición HTTP	1770		1891	449	1770	1
19	16:52:20.036	User 1-70	Petición HTTP	1571		1891	449	1571	1
20	16:52:19.397	User 1-6	Petición HTTP	2329		1891	449	2329	2
21	16:52:19.445	User 1-11	Petición HTTP	2313		1891	449	2312	1
22	16:52:19.406	User 1-7	Petición HTTP	2376		1891	449	2376	1
23	16:52:19.415	User 1-8	Petición HTTP	2367		1891	449	2367	1
24	16:52:19.475	User 1-14	Petición HTTP	2321		1891	449	2320	1
25	16:52:19.986	User 1-65	Petición HTTP	1815		1891	449	1815	1
26	16:52:20.058	User 1-72	Petición HTTP	1773		1891	449	1773	

Figura 3.2: Resultado de la prueba de rendimiento hecha al servicio API RESTful con id API23.

En la herramienta JMeter entonces configuramos el número de solicitudes concurrentes que realizarán la petición, en este caso, se configurará con 1000 solicitudes, después configu-

ramos para que los resultados se puedan apreciar en una tabla, donde se indica la latencia, el tiempo de muestra, el estado, entre otros.

En la Figura 3.2 podemos apreciar el resultado de las 1000 solicitudes hechas a la API23, el estado de todas las solicitudes son exitosas, y el tiempo de muestra promedio, es decir el tiempo en que la solicitud se tarda en responder, es 2428 milisegundos, Tabla 3.7.

Tabla 3.7: Resultados de la prueba de rendimiento del servicio API RESTful con id API23 de la historia de usuario HU010

Muestras	Media	Min	Max	Desv. Estándar	Error	Rendimiento
1000	2428 ms	1239 ms	3191 ms	579.49 ms	0 %	26,6/sec

3.2 CONCLUSIONES

El componente back end del sistema de búsqueda y análisis de vulnerabilidades de dispositivos IoT ha sido desarrollado con éxito y con los resultados esperados, en el proceso de construcción se ha ido implementando buenas prácticas del desarrollo de software. Se diseñó una arquitectura modular y escalable, lo que asegura la mantenibilidad y adaptabilidad futura con la ayuda de la adopción de patrones de diseño como Factory Method y Strategy.

La implementación de medidas de seguridad ha sido fundamental en el proceso de desarrollo del proyecto, haciendo uso de JSON Web Tokens (JWT) hemos asegurado la gestión de privilegios y autenticación en los servicios web, garantizando así una comunicación segura entre los componentes del sistema. La elección de Django como framework principal ha contribuido significativamente a este objetivo, ya que incluye de forma predeterminada varias medidas de seguridad, como el cifrado de contraseñas y la protección contra ataques comunes. Estas medidas han ayudado a garantizar en gran medida la integridad, consistencia y seguridad de los datos.

La comunicación efectiva con el front end se garantizó mediante rigurosas pruebas de funcionalidad y rendimiento. Estas pruebas verificaron que los datos se transmitieran entre los sistemas de manera precisa y eficiente, ayudando también a asegurar la comunicación fluida y sin errores. El uso de índices y consultas óptimas para la base datos mongoDB también ha ayudado a garantizar la efectividad en la comunicación entre el componente back end y el componente front end.

El diseño del sistema se ha logrado gracias a la implementación del modelo C4, permitiendo a todos lo involucrados el rápido entendimiento del sistema al proporcionar una visión clara y comprensible de la arquitectura. La adopción del modelo C4 ha mejorado la comunicación y la colaboración, contribuyendo al éxito del diseño y construcción del sistema.

Los resultados de las pruebas de funcionalidad muestran que los endpoints de los servicios API RESTful creados funcionan correctamente, en ambos casos, casos exitosos y casos fallidos. Las pruebas de rendimiento muestran que el sistema con una cantidad grande de solicitudes es capaz de manejarlas sin errores, en cambio, la desviación estándar junto con los tiempos de respuesta pueden indicar que puede existir mejoras en cuestión de rendimiento.

3.3 RECOMENDACIONES

Un aspecto crucial en el sistema es el almacenamiento y las transacciones que se deben hacer hacia la base de datos, al ser un sistema que prevé manejar información de todos los dispositivos IoT del territorio ecuatoriano la cantidad de almacenamiento y las transacciones son de vital importancia, por lo que es recomendable migrar la base de datos a un sitio con recursos más potentes que los que ofrece de forma gratuita el sitio en la nube en donde está desplegado la base de datos MongoDB actualmente.

Las pruebas de rendimiento indican con el tiempo de respuesta y la desviación estándar que se puede realizar mejoras con respecto al rendimiento, por lo que se recomienda configurar un servidor web para mejorar el comportamiento del sistema contra una cantidad grande de solicitudes. En este proyecto no se realizó por la complejidad y esfuerzo que implica desplegar el sistema en un servidor web como nginx o apache, y por las limitaciones que ofrecen los sitios para desplegar de manera gratuita en la nube.

4 REFERENCIAS BIBLIOGRÁFICAS

[1] L. Pazmiño, F. Flores, L. Ponce et al., «Challenges and Opportunities of IoT Deployment in Ecuador,» en *2019 International Conference on Information Systems and Software Technologies (ICI2ST)*, nov. de 2019, págs. 108-115. DOI: 10.1109/ICI2ST.2019.00022.

[2] *Security Issues in IoT: Challenges and Countermeasures*. dirección: https://www.isaca.org/resources/isaca-journal/issues/2019/volume-1/security-issues-in-iot-challenges-and-countermeasures (visitado 04-12-2023).

[3] *IoT Security Issues, Threats, and Defenses*. dirección: https://www.trendmicro.com/vinfo/us/security/news/internet-of-things/iot-security-101-threats-issues-and-defenses (visitado 04-12-2023).

[4] Diego León, *Tendencias Tecnológicas de Mayor Impacto en el Ecuador 2022 - Evolucionando Digitalmente los Negocios*. dirección: https://www.ey.com/es_ec/consulting/tendencias-tecnologicas-de-mayor-impacto-en-el-ecuador-para-el-a1 (visitado 05-12-2023).

[5] R. Rivera Guevara, «Tools for the detection and analysis of potentially unwanted programs,» Tesis doct., ETSI_Informatica, 2018.

[6] R. Rivera, L. Pazmiño, F. Becerra y J. Barriga, «An analysis of cyber espionage process,» en *Developments and Advances in Defense and Security: Proceedings of MICRADS 2021*, Springer, 2022, págs. 3-14.

[7] Coursera Staff, *What Is the Internet of Things (IoT)? With Examples*. dirección: https://www.coursera.org/articles/internet-of-things (visitado 06-12-2023).

[8] *What is the internet of things (IoT)?* Dirección: https://www.ibm.com/topics/internet-of-things (visitado 06-12-2023).

[9] C. Jiménez y R. Rivera, «Ciberseguridad del IoT: Un Análisis en Países de la Unión Europea,» *Revista Ibérica de Sistemas e Tecnologias de Informação*, n.º E39, págs. 461-476, 2021.

[10] A. Quraishi, M. A. Rusho, A. Prasad, I. Keshta, R. Rivera y M. W. Bhatt, «Employing Deep Neural Networks for Real-Time Anomaly Detection and Mitigation in IoT-Based Smart Grid Cybersecurity Systems,» en *2024 Third International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, IEEE, 2024, págs. 1-6.

[11] *OWASP Internet of Things Project - OWASP*. dirección: https://wiki.owasp.org/index.php/OWASP_Internet_of_Things_Project#tab=IoT_Top_10 (visitado 03-01-2024).

[12] *What is IoT Security? Definition and Challenges of IoT Security*. dirección: https://www.fortinet.com/resources/cyberglossary/iot-security (visitado 03-01-2024).

[13] L. Pazmino, F. Flores, F. Becerra y G. Cevallos, «Challenges and Opportunities of 5G Deployment in Ecuador (extended version),» 2023.

[14] E. Jumbo, J. Llumiquinga, F. Uyaguari, A. Tenezaca, L. Pazmiño y R. Rivera, «Un breve Análisis de Vulnerabilidades en dispositivos IOT en Ecuador,» *Ciencia Latina Revista Científica Multidisciplinar*, vol. 7, n.º 2, págs. 5939-5953, 2023.

[15] *What are ports in computing and how do they work?* Dirección: https://www.techtarget.com/searchnetworking/definition/port (visitado 30-06-2024).

[16] *What is C4 model? - Visual Paradigm Blog*. dirección: https://blog.visual-paradigm.com/what-is-c4-model/ (visitado 06-12-2023).

[17] Scott Rallya, *Mastering Software Architecture Visualization with the C4 Model*. dirección: https://dev.to/fractalis/mastering-software-architecture-visualization-with-the-c4-model-em2 (visitado 06-12-2023).

[18] *The C4 model for visualising software architecture*. dirección: https://c4model.com/ (visitado 06-12-2023).

[19] *What is Django*. dirección: https://aws.amazon.com/es/what-is/django/ (visitado 20-06-2024).

[20] Christiaan Verwijs, Daniel Russo, *A Theory of Scrum Team Effectiveness*. dirección: https://dl.acm.org/doi/full/10.1145/3571849 (visitado 04-12-2023).

[21] *The 2020 Scrum Guide*. dirección: https://scrumguides.org/scrum-guide.html#purpose-of-the-scrum-guide (visitado 08-12-2023).

[22] *Socket Programming in Python (Guide)*. dirección: https://realpython.com/python-sockets// (visitado 30-06-2024).

5 ANEXOS

ANEXO I. PRODUCTO BACKLOG

Tabla 5.1: Product Backlog

ID-HU	HISTORIA DE USUARIO	SPRINT	ESTADO	PRIORIDAD
HU001	Registrar e iniciar sesión	4	TO DO	MEDIA
HU002	Visualizar mi perfil	5	TO DO	BAJA
HU003	Gestionar cuentas	5	TO DO	BAJA
HU004	Visualizar mapa de calor de direcciones IP y puertos	2	TO DO	ALTA
HU005	Escanear completa o parcialmente las direcciones IP	1	TO DO	ALTA
HU006	Re escaneo de un dispositivo	3	TO DO	MEDIA
HU007	Mostrar historial de escaneos	3	TO DO	MEDIA
HU008	Visualizar dashboard	2	TO DO	ALTA
HU009	Mostrar captura de servicios web	4	TO DO	MEDIA
HU010	Mostrar información detallada de vulnerabilidades	1	TO DO	ALTA

ANEXO II. HISTORIAS DE USUARIO

Tabla 5.2: Historia de usuario HU001

Historia de Usuario	
Identificador (ID): HU001	**Usuario**: Administrador – Usuario principal
Nombre Historia: Registrar e iniciar sesión.	
Prioridad de Negocio: Media	**Riesgo de desarrollo**: Bajo
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción: Como administrador deseo que el usuario se registre antes de ingresar al sistema para validar el acceso de cada cuenta y tener un registro de las acciones realizadas dentro del sistema.	
Detalles: La contraseña de los usuarios deben estar cifrados, para evitar problemas de seguridad.	
Criterios de aceptación: ❒ En cada campo debe existir una validación de los datos ingresados dependiendo del tipo de campo (correo electrónico, contraseña, etc.), en el caso de la contraseña se debe validar que esta cumpla con los requisitos típicos (longitud mínima, uso de caracteres especiales, etc.). ❒ Se debe dar un mensaje sobre el problema específico que exista al ingresar información incorrecta en un campo. ❒ Al almacenar la contraseña en la base de datos esta debe cifrarse con algún algoritmo de encriptación. ❒ Debe existir una confirmación del correo electrónico a través del envío de un correo con el enlace de confirmación. ❒ El usuario debe ser capaz de iniciar sesión correctamente con sus credenciales una vez registrado. ❒ Debe existir un típico sistema de “¿Olvidó su contraseña?” para que el usuario pueda restablecer su contraseña en el envío de un mensaje a su correo electrónico.	

Tabla 5.3: Historia de usuario HU002

Historia de Usuario	
Identificador (ID): HU002	**Usuario**: Usuario principal
Nombre Historia: Visualizar mi perfil.	
Prioridad de Negocio: Baja	**Riesgo de desarrollo**: Bajo
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción: Como usuario quiero visualizar los datos de mi perfil y escaneos realizados para agilizar el proceso de revisar y gestionar mis búsquedas, brindándome una experiencia más rápida y personalizada.	
Detalles:	
Criterios de aceptación: ❒ Para poder acceder a su información, el usuario deberá iniciar previamente su sesión. ❒ Se debe mostrar un listado con las búsquedas realizadas y almacenadas por el usuario. ❒ El usuario podrá guardar o eliminar los escaneos que realiza mediante un botón. ❒ La página de perfil debe mostrar de manera clara y organizada, la información personal del usuario, como nombre, dirección de correo electrónico, fecha de registro, entre otros datos relevantes. ❒ El usuario podrá editar su información de perfil y actualizar su contraseña.	

Tabla 5.4: Historia de usuario HU003

Historia de Usuario	
Identificador (ID): HU003	**Usuario**: Administrador
Nombre Historia: Gestionar cuentas.	
Prioridad de Negocio: Baja	**Riesgo de desarrollo**: Bajo
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción: Como administrador deseo que las cuentas del usuario puedan gestionarse para tener control de estas, logrando así, activar y desactivar cada una de las cuentas.	
Detalles:	

Criterios de aceptación:

- ❒ El administrador deberá ser capaz de activar o desactivar una cuenta de un usuario registrado en cualquier momento.
- ❒ Al momento de activar o desactivar una cuenta el usuario podrá contar o no, respectivamente, con los servicios disponibles que ofrece el sistema.

Tabla 5.5: Historia de usuario HU004

Historia de Usuario	
Identificador (ID): HU004	**Usuario**: Usuario principal
Nombre Historia: Visualizar mapa de calor de direcciones IP y puertos.	
Prioridad de Negocio: Alta	**Riesgo de desarrollo**: Medio
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción: Como usuario quiero visualizar a manera de mapa geográfico el número de vulnerabilidades existentes en cada provincia del país, para poder identificar las áreas con mayor riesgo de seguridad.	
Detalles:	
Criterios de aceptación: ❒ Según el número de dispositivos vulnerables en cada provincia se tendrá un color que variará entre verde, amarillo y rojo. ❒ La información del mapa debe actualizarse de manera frecuente, a fin de reflejar el estado de vulnerabilidad en cada provincia. ❒ Debe proporcionarse una leyenda clara que explique el significado de cada color asociado a los niveles de riesgo. ❒ Hay que asegurar que la visualización del mapa sea compatible con dispositivos móviles, como tabletas y teléfonos, para que los usuarios puedan acceder a la información de manera conveniente.	

Tabla 5.6: Historia de usuario HU005

Historia de Usuario	
Identificador (ID): HU005	**Usuario**: Administrador
Nombre Historia: Escanear completa o parcialmente las direcciones IP.	
Prioridad de Negocio: Alta	**Riesgo de desarrollo**: Medio
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción: Como administrador, deseo contar con la capacidad de realizar escaneos completos o parciales de las direcciones IP dentro del territorio ecuatoriano para actualizar la información sobre el rango de direcciones IP asociadas a los dispositivos IoT identificados.	
Detalles: Esto es necesario para que únicamente el administrador sea capaz de realizar un escaneo de varias direcciones IP y actualizar la información de la página, a diferencia del usuario que podrá realizar la actualización de solo una dirección a la vez con el fin de no exceder el consumo recursos computacionales.	
Criterios de aceptación: ❒ El administrador deberá poder elegir el rango que desea escanear. ❒ El administrador podrá filtrar los rangos de escaneo por provincia. ❒ Si el administrador elige el escaneo de un rango específico de direcciones IP el sistema debe validar si este rango se encuentra dentro del rango total de direcciones IP del territorio ecuatoriano. ❒ Una vez escaneados los puertos estos deben actualizarse en la base de datos y a su vez en la página web.	

Tabla 5.7: Historia de usuario HU006

Historia de Usuario	
Identificador (ID): HU006	**Usuario**: Usuario principal
Nombre Historia: Re escaneo de un dispositivo.	
Prioridad de Negocio: Media	**Riesgo de desarrollo**: Medio
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	

Descripción: Como usuario quiero actualizar la información de las vulnerabilidades de dispositivos IoT mediante un re escaneo, para conocer si las vulnerabilidades se han mitigado, han aumentado o siguen igual.
Detalles: El usuario será capaz de actualizar la información de una dirección IP a la vez con el fin de no exceder el consumo de recursos.
Criterios de aceptación: ❒ El usuario solo podrá realizar el escaneo de una dirección IP a la vez. ❒ Una vez terminado el escaneo los datos deberán actualiza en la base de datos y en la página web.

Tabla 5.8: Historia de usuario HU007

Historia de Usuario	
Identificador (ID): HU007	**Usuario**: Usuario principal
Nombre Historia: Mostrar historial de escaneos.	
Prioridad de Negocio: Media	**Riesgo de desarrollo**: Medio
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción: Como usuario quiero visualizar todos los escaneos anteriores que se realizaron a un dispositivo IoT para poder conocer los cambios de estado o vulnerabilidades que pueden existir en cada escaneo.	
Detalles:	
Criterios de aceptación: ❒ La información de cada dispositivo debe incluir tarjetas con información de escaneos anteriores. ❒ La información debe incluir la fecha y hora del escaneo y su número de vulnerabilidades.	

Tabla 5.9: Historia de usuario HU008

Historia de Usuario	
Identificador (ID): HU008	**Usuario**: Usuario principal
Nombre Historia: Visualizar dashboard.	
Prioridad de Negocio: Alta	**Riesgo de desarrollo**: Medio
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción: Como usuario quiero visualizar estadísticas de manera gráfica información relevante de los dispositivos y sus vulnerabilidades, en el Ecuador o por cada provincia.	
Detalles: Se mostrará la información en base a puertos, estados y provincias.	
Criterios de aceptación: ❒ Los datos proporcionados deben presentarse de manera gráfica. ❒ Permitir la interactividad con los gráficos para que el usuario pueda obtener detalles adicionales al hacer clic en una provincia. ❒ Asegurar que los gráficos se actualicen constantemente para reflejar cualquier cambio en el número de dispositivos vulnerables. ❒ Incluir leyendas claras y descriptivas en los gráficos para que el usuario pueda entender fácilmente la información presentada.	

Tabla 5.10: Historia de usuario HU009

Historia de Usuario	
Identificador (ID): HU009	**Usuario**: Usuario principal
Nombre Historia: Mostrar captura de servicios web.	
Prioridad de Negocio: Media	**Riesgo de desarrollo**: Bajo
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción: Como usuario deseo tener una captura del servicio web correspondiente a cada dispositivo IoT, de ser el caso, para conocer el tipo de servicio que se encuentra vulnerable.	
Detalles: Para que el usuario pueda acceder a las capturas de los servicios se debe realizar una solicitud al administrador, esperando una respuesta.	

Criterios de aceptación:

- ❒ Las cuentas de usuario deben crearse por defecto sin este permiso.
- ❒ El usuario deberá ser capaz de realizar una solicitud para habilitar el permiso de mostrar las capturas de los posibles servicios web.
- ❒ El proceso de solicitud solamente estará activo si el usuario no tiene estos permisos.
- ❒ Al momento que el administrador habilita el permiso, en la cuenta de usuario respectiva, se podrá apreciar dichas capturas.

Tabla 5.11: Historia de usuario HU010

Historia de Usuario	
Identificador (ID): HU010	**Usuario**: Usuario principal
Nombre Historia: Mostrar información detallada de vulnerabilidades.	
Prioridad de Negocio: Alta	**Riesgo de desarrollo**: Alto
Iteración Asignada:	
Responsable: Mike Encalada – Isaí Saransig	
Descripción:Como usuario quiero obtener información detallada al buscar una dirección IP, para conocer más a fondo los detalles del escaneo realizado en cada dispositivo.	
Detalles:	
Criterios de aceptación: ❒ Proporcionar una vista detallada que incluya información específica del escaneo asociado a la dirección IP seleccionada, como fecha y hora del escaneo, posibles vulnerabilidades y cualquier otra información relevante. ❒ Asegurar que la información detallada se pueda presentar adecuadamente en dispositivos móviles para facilitar su consulta en diferentes plataformas.	

ANEXO III. CASOS DE PRUEBA (PRUEBAS DE FUNCIONALIDAD)

Tabla 5.12: Caso de prueba CP_API_001

Identificador del Caso de Prueba	CP_API_001
Nombre del Caso de Prueba	Validar respuestas del endpoint GET /lista_dispositivos/iot_device/<str:ip>/ con parámetros válidos en inválidos
Descripción	Este caso de prueba verifica que el endpoint GET /lista_dispositivos/iot_device/<str:ip>/ maneja correctamente tanto solicitudes exitosas como solicitudes con errores, devolviendo la información de un dispositivo IoT.
Precondiciones	❒ El servidor de la API debe estar en funcionamiento. ❒ Puede haber o no el dispositivo con su dirección IP registrada en la base de datos.
Datos de Entrada	❒ Escenario de Éxito: ✧ Método HTTP: GET ✧ Segmento URL: /lista_dispositivos/iot_device/201.234.193.196/ ✧ Razón de éxito: Se coloca una dirección IP con un formato válido y que no pertenece a Ecuador ✧ Encabezados: `Authorization: Bearer <token>` ❒ Escenario de Error 1: ✧ Método HTTP: GET ✧ Segmento URL: /lista_dispositivos/iot_device /7.0.2.4.6/ (parámetro inválido) ✧ Razón de error: Se coloca una dirección IP con un formato inválido. ✧ Encabezados: `Authorization: Bearer <token>`

Datos de Entrada	❒ Escenario de Error 2: ✧ Método HTTP: GET ✧ Segmento URL: /lista_dispositivos/iot_device /8.51.10.0/ (parámetro inválido) ✧ Razón de error: Se coloca una dirección IP con un formato válido pero que no pertenece a Ecuador. ✧ Encabezados: `Authorization: Bearer <token>`

Datos Esperados	❒ Escenario de Éxito:
	✧ Código de estado HTTP: 200 OK
	✧ Estructura de la respuesta:
	`{`
	`"ip_address": "201.234.193.196",`
	`"status": "Activo",`
	`"created": "2024-05-24T20:03:41.481000",`
	`"updated": "2024-07-03T19:37:49.995000",`
	`"city": "Quito",`
	`.....`
	`"vulnerability":[...]`
	`}`
	
	❒ Escenario de Error 1:
	✧ Código de estado HTTP: 400 Bad Request
	✧ Estructura de la respuesta:
	`{`
	`"error": "La IP no es valida"`
	`}`
	
	❒ Escenario de Error 2:
	✧ Código de estado HTTP: 400 Bad Request
	✧ Estructura de la respuesta:
	`{`
	`"error": "La IP no pertenece a Ecuador"`
	`}`

Resultados Esperados	❒ Escenario de Éxito: La API devuelve un array JSON con la información del dispositivo IoT si se encuentra en la base de datos, de no encontrarse realiza el escaneo para guardar y obtener la información. ❒ Escenario de Error 1: La API devuelve un mensaje de error explicando que el valor de la dirección IP pasado como parámetro es inválido. ❒ Escenario de Error 2: La API devuelve un mensaje de error explicando que la dirección IP pasado como parámetro no pertenece a Ecuador.
Notas	❒ Asegurarse de que el token de autorización es válido, es decir, que pertenezca a una cuenta de usuario válida.

Tabla 5.13: Caso de prueba CP_API_002

Identificador del Caso de Prueba	CP_API_002
Nombre del Caso de Prueba	Validar respuestas del endpoint GET /dashboard/-get_dashboard_api/<str:provincia>/ con parámetros válidos en inválidos
Descripción	Este caso de prueba verifica que el endpoint GET /dashboard/-get_dashboard_api/<str:provincia>/ maneja correctamente tanto solicitudes exitosas como solicitudes con errores, devolviendo la información correcta de los diagramas del dashboard.
Precondiciones	❒ El servidor de la API debe estar en funcionamiento. ❒ Debe existir por lo menos 1 dispositivo registrado en la base de datos.
Datos de Entrada	❒ Escenario de Éxito 1: ✧ Método HTTP: GET ✧ Segmento URL: /dashboard/get_dashboard_api/Pichincha/ ✧ Razón de éxito: Se coloca una provincia válida del territorio ecuatoriano. ✧ Encabezados: `Authorization: Bearer <token>`

Datos de entrada	❒ Escenario de Éxito 2: ✧ Método HTTP: GET ✧ Segmento URL: /dashboard/get_dashboard_api/ ✧ Razón de éxito: Al no colocarse como parámetro una provincia la información devuelta deberá ser con respecto a todo el Ecuador. ✧ Encabezados: `Authorization: Bearer <token>` ❒ Escenario de Error: ✧ Método HTTP: GET ✧ Segmento URL: /dashboard/get_dashboard_api/EstadosUnidos/ (parámetro inválido) ✧ Razón de error: Se coloca una provincia que no existe en la base de datos. ✧ Encabezados: `Authorization: Bearer <token>`

Datos Esperados	❒ Escenario de Éxito 1 y 2:
	✧ Código de estado HTTP: 200 OK
	✧ Estructura de la respuesta:
	`[`
	`{`
	`"vulnerables": 11,`
	`"status": {`
	`"Activo": 11`
	`},`
	`"ports": {`
	`"22": 2,`
	`...`
	`"8443": 2`
	`}`
	`}`
	`]`
	❒ Escenario de Error:
	✧ Código de estado HTTP: 200 OK
	✧ Estructura de la respuesta:
	`{`
	`"status": {},`
	`"ports": {}`
	`}`

Resultados Esperados	❒ Escenario de Éxito 1: La API devuelve un array JSON con la información de los dispositivos perteneciente a una provincia, en este caso, Pichincha. ❒ Escenario de Éxito 2: La API devuelve un array JSON con la información de los dispositivos de todo el territorio ecuatoriano ❒ Escenario de Error: La API devuelve un JSON con información vacía.
Notas	❒ Asegurarse de que el token de autorización es válido, es decir, que pertenezca a una cuenta de usuario válida.

Tabla 5.14: Caso de prueba CP_API_003

Identificador del Caso de Prueba	CP_API_003
Nombre del Caso de Prueba	Validar respuestas del endpoint GET /escaneo/screenshot/<str:ip>/<int:port>/ con parámetros válidos en inválidos
Descripción	Este caso de prueba se verifica que el endpoint GET /escaneo/screenshot/<str:ip>/<int:port>/ maneja correctamente tanto solicitudes exitosas como solicitudes con errores, devolviendo la url para obtener la captura de pantalla.
Precondiciones	❒ El servidor de la API debe estar en funcionamiento. ❒ El dispositivo IoT debe tener abierto o activo el puerto 80, 8080 o 443. ❒ El token debe pertenecer a un usuario con permiso de captura, es decir, un usuario que pertenece al grupo respectivo.
Datos de Entrada	❒ Escenario de Éxito: ✧ Método HTTP: GET ✧ Segmento URL: /escaneo/screenshot/201.234.193.196/80/ ✧ Razón de éxito: Se coloca una IP válida y un puerto que el dipositivo tiene abierto y cuenta con un servicio web activo. ✧ Encabezados: `Authorization: Bearer <token>`

Datos de entrada	❒ Escenario de Error: ✧ Método HTTP: GET ✧ Segmento URL: /escaneo/screenshot/201.234.193.196/54/ ✧ Razón de error: Se coloca un puerto diferente a los puertos 80, 8080 o 443. ✧ Encabezados: `Authorization: Bearer <token>`

Datos Esperados	❒ Escenario de Éxito:
	✧ Código de estado HTTP: 201 Created
	✧ Estructura de la respuesta:
	`{`
	`"screenshot_url": "http://localhost:8000/media/screenshots/201.234.193.196_80_http.png"`
	`}`
	
	❒ Escenario de Error:
	✧ Código de estado HTTP: 400 Bad Request
	✧ Estructura de la respuesta:
	`{`
	`"error": "El puerto enviado no es correcto"`
	`}`

Resultados Esperados	❒ Escenario de Éxito 1: La API devuelve un array JSON con la url para acceder a la captura de pantalla del servicio web. ❒ Escenario de Error: La API devuelve un JSON con un mensaje de error indicando que el puerto es incorrecto.
Notas	❒ Asegurarse de que el token de autorización es válido, es decir, que pertenezca a una cuenta de usuario válida.

Tabla 5.15: Caso de prueba CP_API_004

Identificador del Caso de Prueba	CP_API_004
Nombre del Caso de Prueba	Validar el correcto registro de un usuario.
Descripción	Este caso de prueba se verifica que el endpoint POST /autenticacion/register/ maneja correctamente tanto solicitudes exitosas como solicitudes con errores, registrando a un usuario en el sistema.
Precondiciones	❒ El servidor de la API debe estar en funcionamiento.
Datos de Entrada	❒ Escenario de Éxito: ✧ Método HTTP: POST ✧ Segmento URL: /autenticacion/register/ ✧ Cuerpo: ○ Estructura del cuerpo: `{ "username": "MikeAngelo", "email": "angelomike653@gmail.com", "password": "tes*****d*76", "first_name": "Mike", "last_name": "Encalada", "organization": "EPN", "role": "Student" }` ✧ Razón de éxito: Cada uno de los parámetros son correctos.

Datos de entrada	❒ Escenario de Error: ✧ Método HTTP: POST ✧ Segmento URL: /autenticacion/register/ ✧ Cuerpo: ○ Estructura del cuerpo: `{ "username": "MikeAngelo", "email": "angelomike654@gmail.com", "password": "tes*****d*76", "first_name": "Mike", "last_name": "Encalada", "organization": "EPN", "role": "Student" }` ✧ Razón de error: Se registra un usuario con un username ya registrado anteriormente.

Datos Esperados	❒ Escenario de Éxito:
	✧ Código de estado HTTP: 200 OK
	✧ Estructura de la respuesta:
	`{ "success": "Registro exitoso" }`
	❒ Escenario de Error:
	✧ Código de estado HTTP: 500 Internal Server Error
	✧ Estructura de la respuesta:
	`{ "error": "El usuario ya existe" }`

Resultados Esperados	❒ Escenario de Éxito: La API devuelve un array JSON indicando que se realizó el registro correctamente. ❒ Escenario de Error: La API devuelve un JSON con un mensaje de error indicando que ese usuario ya se encuentra registrado.

Notas	❒ El sistema se debe encargar de codificar la contraseña antes de guardarla en la base de datos.

Tabla 5.16: Caso de prueba CP_API_005

Identificador del Caso de Prueba	CP_API_005
Nombre del Caso de Prueba	Validar el correcto inicio de sesión del usuario.
Descripción	Este caso de prueba se verifica que el endpoint POST /api/token/ maneja correctamente tanto solicitudes exitosas como solicitudes con errores, autenticando e iniciando sesión.
Precondiciones	❒ El servidor de la API debe estar en funcionamiento.

Datos de Entrada	❒ Escenario de Éxito:
	✧ Método HTTP: POST
	✧ Segmento URL: /api/token/
	✧ Cuerpo:
	○ Estructura del cuerpo:
	`{`
	`"username": "MikeAngelo",`
	`"password": "tes****ord*76",`
	`}`
	✧ Razón de éxito: Cada uno de los parámetros son correctos, el usuario existe y la contraseña es correcta

Datos de entrada	❒ Escenario de Error:
	✧ Método HTTP: POST
	✧ Segmento URL: /api/token/
	✧ Cuerpo:
	○ Estructura del cuerpo:
	`{`
	`"username": "MikeAngelo",`
	`"password": "pass****est*76",`
	`}`
	✧ Razón de error: Se intenta iniciar sesión con una contraseña incorrecta.

Datos Esperados	❒ Escenario de Éxito:
	✧ Código de estado HTTP: 200 OK
	✧ Estructura de la respuesta:
	`{`
	`"refresh": "eyJhbGciOiJIUzI1NiIsInR...",`
	`"access": "eyJhbGciOiJIUzI1NiIsInR5..."`
	`}`
	❒ Escenario de Error:
	✧ Código de estado HTTP: 401 Unauthorized
	✧ Estructura de la respuesta:
	`{`
	`"detail": "No active account found with the given credentials"`
	`}`

Resultados Esperados	❒ Escenario de Éxito: La API devuelve un array JSON con el token de acceso y el token para actualizar el token de acceso. ❒ Escenario de Error: La API devuelve un JSON con un mensaje de error indicando no que existe cuentas activas con esas credenciales.

ANEXO IV. REPOSITORIO GENERAL DEL SISTEMA

Enlace al repositorio: https://github.com/isaisaransig/SecurityScanIoT

ANEXO V. REPOSITORIO DE LA RAMA DEL BACK END

Enlace al repositorio: https://github.com/isaisaransig/SecurityScanIoT/tree/MikeDev

ANEXO VI. MOCKUP HECHO EN FIGMA

Enlace al prototipo: https://www.figma.com/file/FwiHOgcmT3v3E4x88FFXcW/Tesis-Design?type=design&node-id=0-1&mode=design

ANEXO VII. TAREAS DE LAS HISTORIAS DE USUARIO

Enlace a las tareas: https://epnecuador-my.sharepoint.com/:w:/g/personal/mike_encalada_epn_edu_ec/EQqVhogEostCtB_MbYU6Ft8B78_qtQ3vBE4HZ_9U4mUfjw?e=uuOdbi

ANEXO VIII. MODELO C4

Enlace a los diagramas: https://epnecuador-my.sharepoint.com/:f:/g/personal/mike_encalada_epn_edu_ec/EqRzi9H6uHIEtyrEo9mEDAsBd5ywQERH7PlcizR5gpEMgQ?e=6xvmxw

ANEXO IX. VIDEO DEMOSTRATIVO DEL SISTEMA

Enlace al video: https://youtu.be/W-hCNQE9BUs